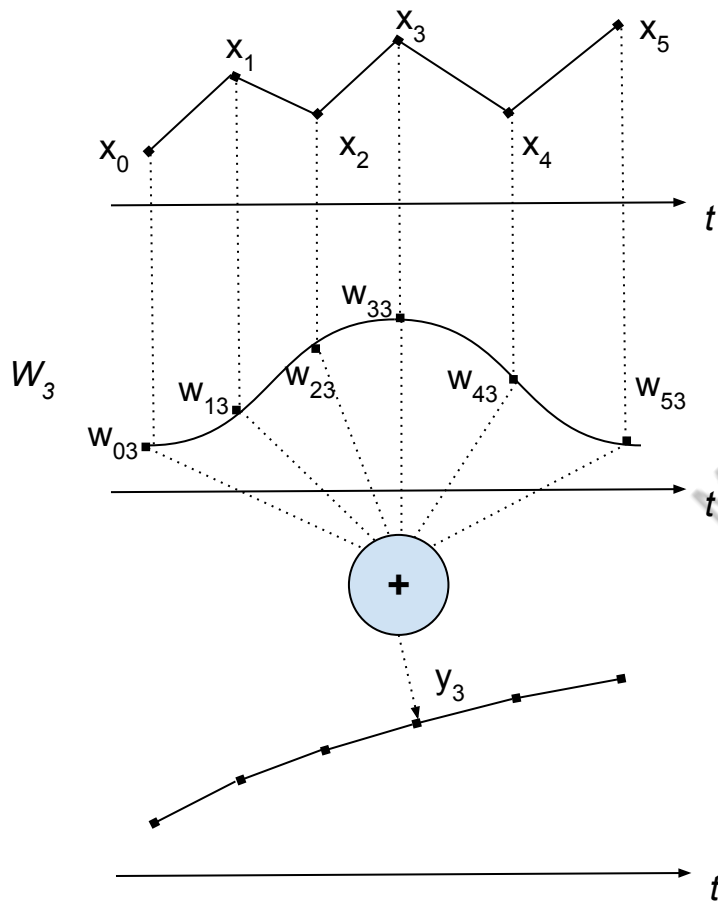




ATTENTION!

How to smooth a noisy time series



Noisy time series data: $x = [x_0, x_1, x_2, x_3, x_4, x_5]$

Re-weighting function W_3 for x_3 :

The normal distribution like function W_3 will filter data that's far away of x_3 and amplifies data that's close

$$y_3 = W_3^T \cdot x, y_4 = W_4^T \cdot x \dots$$

$$y = [y_0, y_1, y_2, y_3, y_4, y_5]$$

Compared with x , y 's data points have other's **context**, y_3 paid attention to other points

Word Embedding

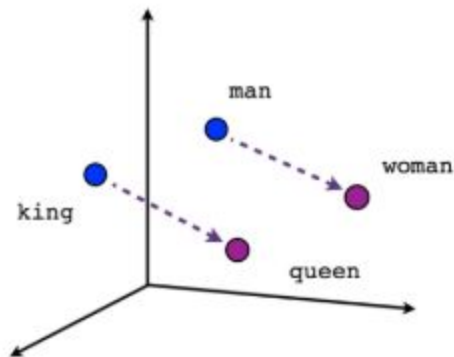
Definition: a word embedding is a representation of a word. The embedding is used in text analysis. Typically, the representation is a real-valued vector that encodes the meaning of the word in such a way that the words that are closer in the vector space are expected to be similar in meaning.

Example:

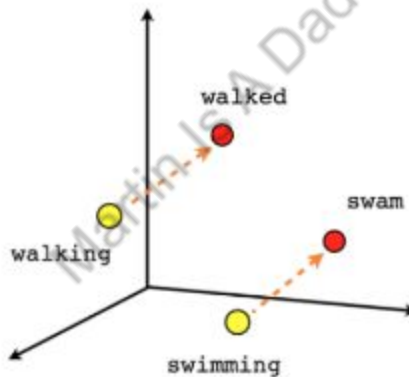
- Embedding definition of vector: {is_human, is_female, is_male, bark_or_not, talk_or_not}.
- Representation of **`father`**: {1, 0, 1, 0, 1}
- Representation of **`stud`**: {0, 0, 1, 1, 0}

Word Embedding

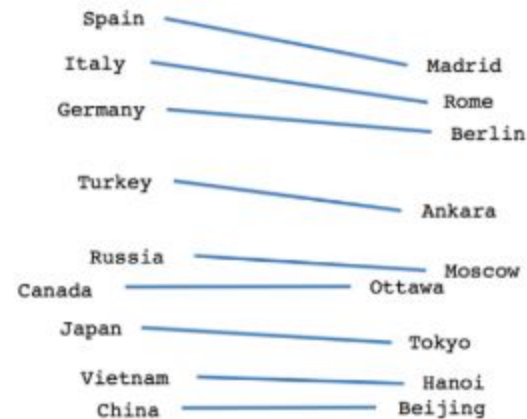
The distance between vectors indicates the similarity between words



Male-Female



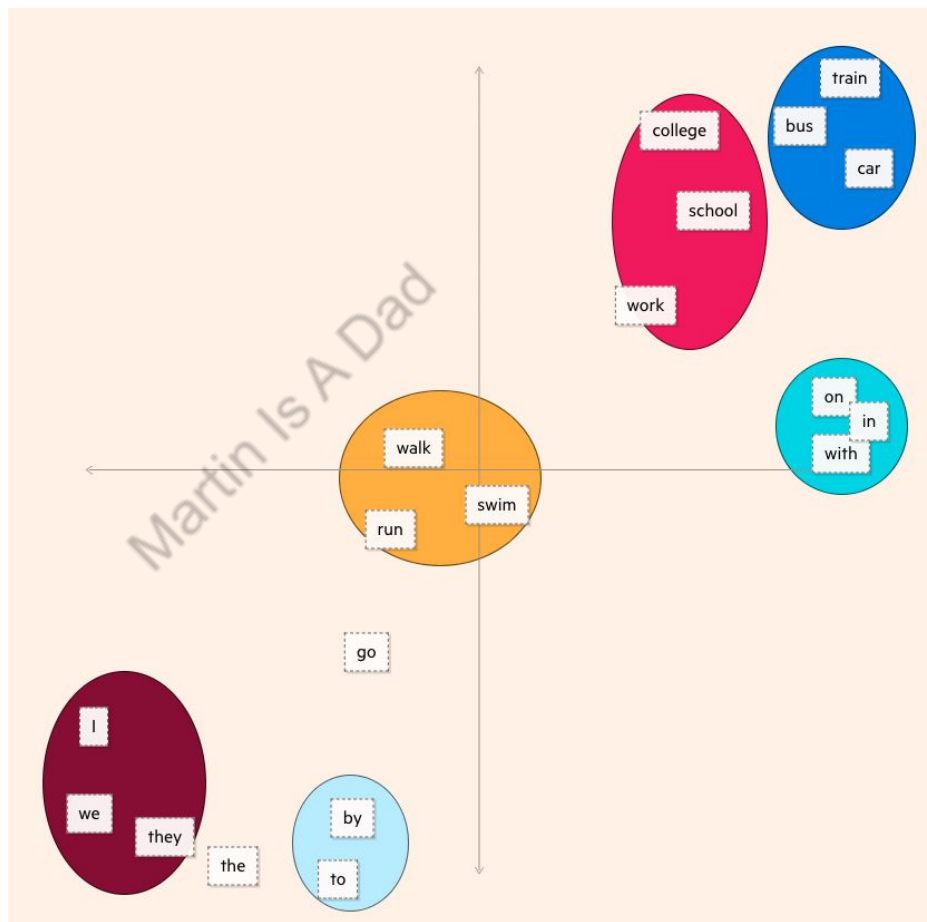
Verb tense



Country-Capital

Word Embedding

Words with similar meanings are represented by similar vectors



Attention for natural language

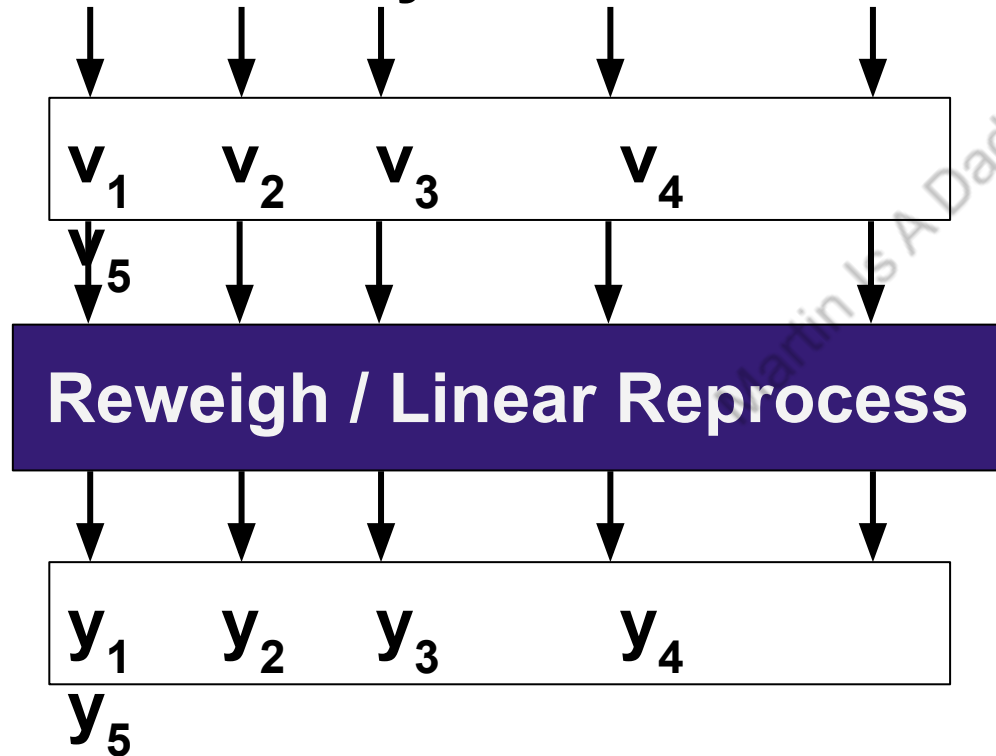
Date is my favorite fruit



- Imagine natural language needs similar mechanism, each word/token/data point needs context of others. For example, `Date` in this sentence means an edible fruit instead of time, with context provided by `fruit`.
- However we can not blindly use proximity like we did. We need a more sophisticated reweighing method related to the semantics of natural language.

Attention for natural language

Date is my favorite fruit



v_1 is the word embedding of 'Date', v_2 is the word embedding of 'is' etc

y_1 should be a better representation than v_1 given it has more context of other words

Attention for natural language

Date is my favorite fruit

$\mathbf{v}_1$ $\mathbf{v}_2$ $\mathbf{v}_3$ $\mathbf{v}_4$

$\mathbf{v}_5$

Reweight / Linear Reprocess

$\mathbf{y}_1$ $\mathbf{y}_2$ $\mathbf{y}_3$ $\mathbf{y}_4$

$\mathbf{y}_5$

$$\begin{aligned} \mathbf{v}_1^T \cdot \mathbf{v}_1 &= w_{11}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_2 &= w_{12}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_3 &= w_{13}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_4 &= w_{14}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_5 &= w_{15}^* \end{aligned}$$

Normalize

$$\begin{aligned} w_{11} \\ w_{12} \\ w_{13} \\ w_{14} \\ w_{15} \end{aligned}$$

$$\text{where } w_{11} + w_{12} + w_{13} + w_{14} + w_{15} = 1$$

$$w_{11} \cdot \mathbf{v}_1 + w_{12} \cdot \mathbf{v}_2 + w_{13} \cdot \mathbf{v}_3 + w_{14} \cdot \mathbf{v}_4 + w_{15} \cdot \mathbf{v}_5 = \mathbf{y}_1$$

(Reweights all vector towards $\mathbf{v}_1$ so it has context of other words' embedding)

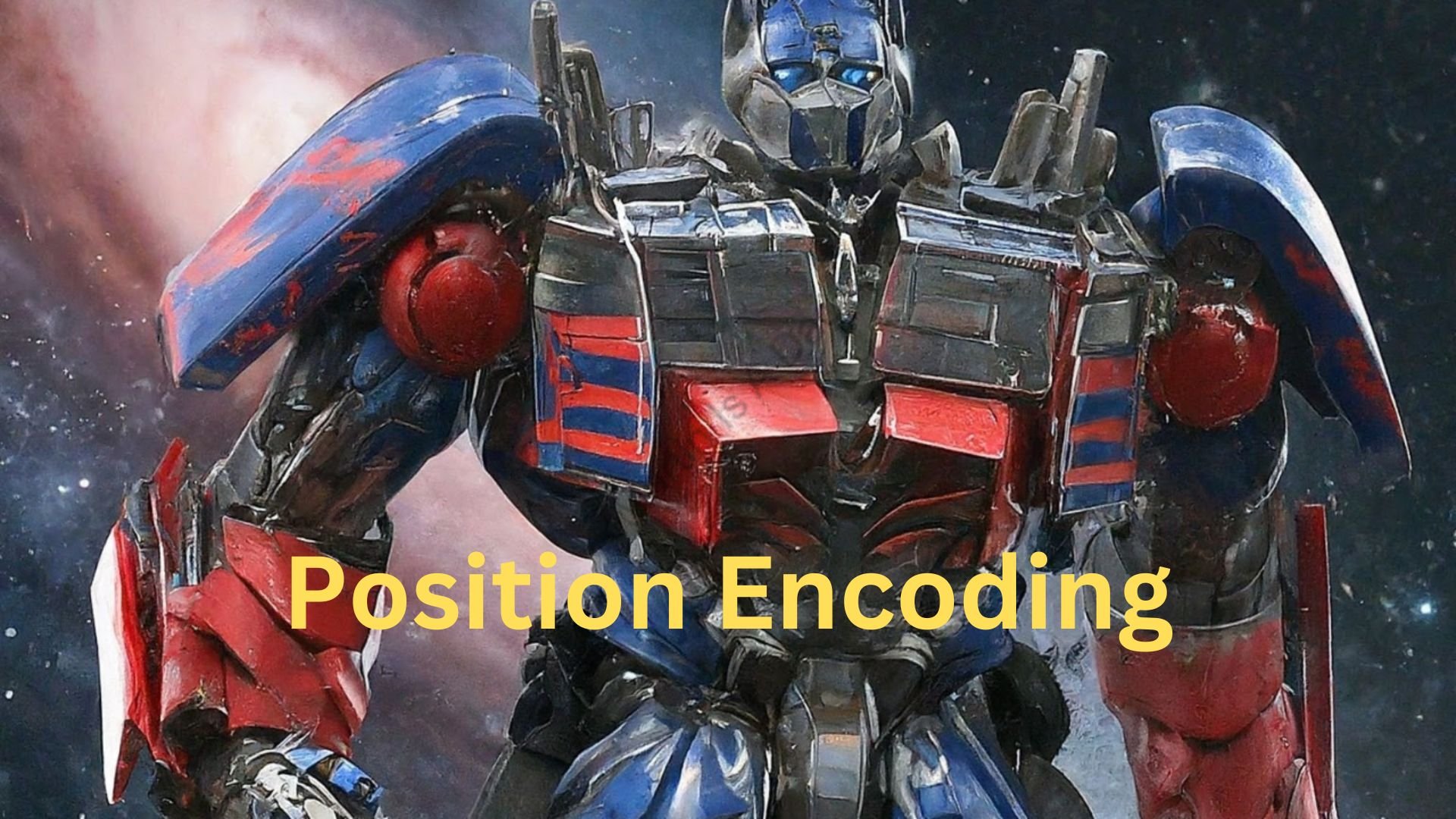
Similarly:

$$w_{21} \cdot \mathbf{v}_1 + w_{22} \cdot \mathbf{v}_2 + w_{23} \cdot \mathbf{v}_3 + w_{24} \cdot \mathbf{v}_4 + w_{25} \cdot \mathbf{v}_5 = \mathbf{y}_2$$

$$w_{31} \cdot \mathbf{v}_1 + w_{32} \cdot \mathbf{v}_2 + w_{33} \cdot \mathbf{v}_3 + w_{34} \cdot \mathbf{v}_4 + w_{35} \cdot \mathbf{v}_5 = \mathbf{y}_3$$

$$w_{41} \cdot \mathbf{v}_1 + w_{42} \cdot \mathbf{v}_2 + w_{43} \cdot \mathbf{v}_3 + w_{44} \cdot \mathbf{v}_4 + w_{45} \cdot \mathbf{v}_5 = \mathbf{y}_4$$

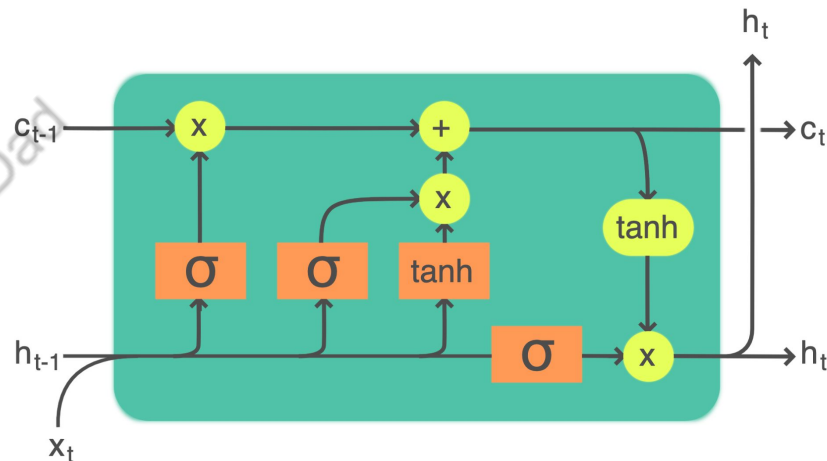
$$w_{51} \cdot \mathbf{v}_1 + w_{52} \cdot \mathbf{v}_2 + w_{53} \cdot \mathbf{v}_3 + w_{54} \cdot \mathbf{v}_4 + w_{55} \cdot \mathbf{v}_5 = \mathbf{y}_5$$

A detailed image of Optimus Prime, the leader of the Autobots, standing against a dark, starry background. He is a large, blue and red robot with a prominent red visor and a blue head. His chest features a red and blue striped pattern. The text "Position Encoding" is overlaid in a bold, yellow font across the center of his torso.

Position Encoding

Before Transformer: LSTM

- A traditional RNN has a single hidden state that is passed through time, which can make it difficult for the network to learn long-term dependencies. **LSTMs model** address this problem by introducing a memory cell, which is a container that can hold information for an extended period.
- LSTM architectures are capable of learning long-term dependencies in sequential data, which makes them well-suited for tasks such as language translation, speech recognition, and time series forecasting.
- **However, LSTMs are slower and less efficient than Transformers due to their sequential nature.**



Transformer to the Rescue

- Transformers can capture global dependencies more effectively in long sequences compared to LSTMs and GRUs.
- Transformer is better at parallelizing compute

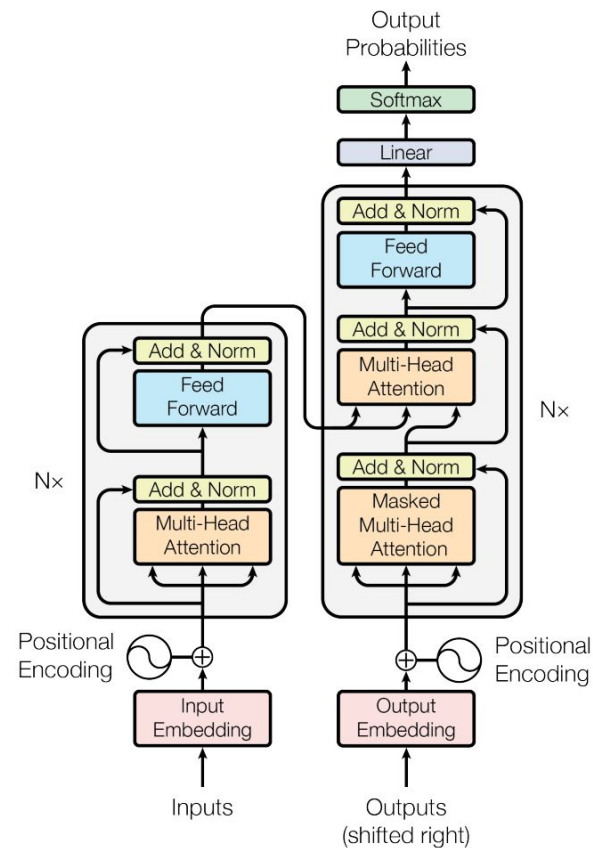


Figure 1: The Transformer - model architecture.

Transformer: Self Attention

As discussed in previous episodes, Self Attention mechanism captures dependencies and context

Date is my favorite fruit



A diagram illustrating the self-attention mechanism. The sentence "Date is my favorite fruit" is shown. The word "Date" is blue and underlined, and the word "fruit" is green and underlined. A blue line connects the bottom of "fruit" to the bottom of "Date", with an upward-pointing arrow at the "Date" end, indicating a dependency or context captured by the self-attention mechanism.

Transformer: Position Encoding

Order Matters!

Here is why order matters

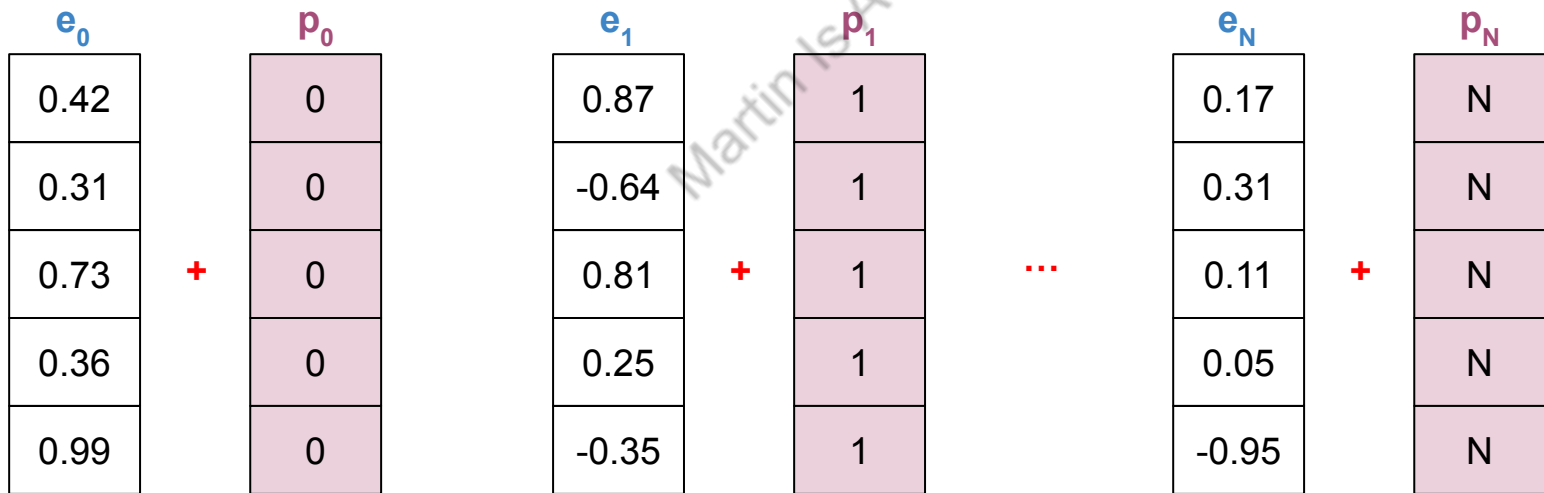
Even though she did **not** win the award, she was satisfied.

Even though she did win the award, she was **not** satisfied.

We need position encoding!

Position Encoding: Naive Approach?

- Not working since large N will skew all the embeddings
- Need the following properties:
 - Bound min and max values
 - Still be able to differentiate encoded position info



Position Encoding: Naive Approach + Normalization?

- Not working since:
 - Encoding for the same word combination will have different encodings if the sentence length is different.
 - Position encoding for 'I am' will be different for 'I am a dad' and 'I am Martin'
- Need the following property:
 - Ideally not dependent on the length of the sentence.

e_0		p_0		e_1		p_1		e_N		p_N
0.42		0		0.87		$1/N$		0.17		1
0.31		0		-0.64		$1/N$		0.31		1
0.73	+	0		0.81	+	$1/N$	...	0.11	+	1
0.36		0		0.25		$1/N$		0.05		1
0.99		0		-0.35		$1/N$		-0.95		1

Position Encoding

- pos: position of the word
- d: dimension of the embedding for words
- $2i$: even indices of PE value within d dimensions
- $2i+1$: odd indices of PE value within d dimensions

Now we combined semantic embedding and position encoding, this is positionally-aware semantic embeddings!

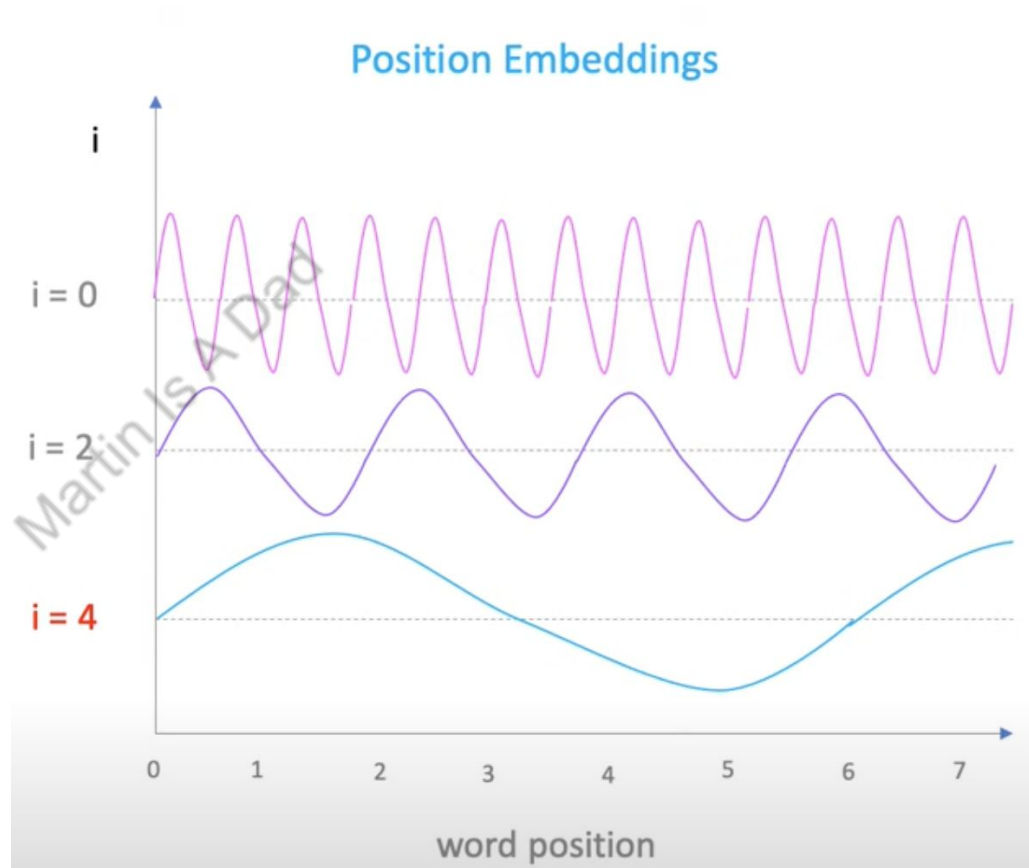
e_0	p_0	e_1	p_0	e_2	p_0	e_3	p_0
0.42	0.82	0.87	-0.33	0.02	0.27	0.38	-0.78
0.31	0.79	-0.64	-0.25	0.01	0.39	0.16	-0.87
0.73	0.76	0.81	-0.17	-0.24	0.50	0.01	-0.94
0.36	0.74	0.26	-0.09	-0.07	0.60	0.09	-0.98
0.99	0.71	-0.35	-0.02	0.00	0.69	0.00	-1.00

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

Position Encoding - Intuition

- Bound min and max values
- Able to differentiate encoded position info
- Not dependent on the length of the sentence.



A detailed image of Optimus Prime, the leader of the Autobots, standing against a dark, starry background. He is a large, blue and red robot with a prominent red visor and a blue helmet. His chest features a red and blue striped pattern. The text "Keys Values Queries" is overlaid in a bold, yellow font across the center of the image.

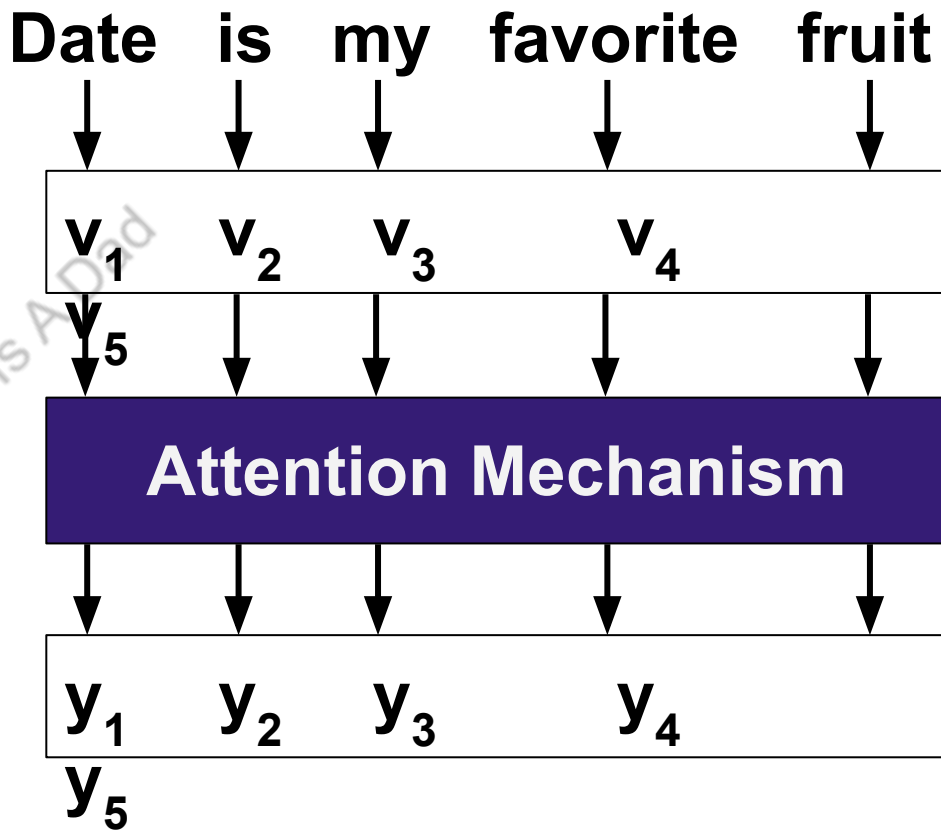
Keys Values Queries

Recap on Simplified Attention Mechanism

- $\mathbf{y}$ has more context taken into consideration compared with $\mathbf{v}$, including position encoding
- This is achieved by attention mechanism

Now let's dive deeper into what actually happened

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \in \mathbb{R}^{m \times d_v}$$



Recap on Simplified Attention Mechanism

Date is my favorite fruit

$\mathbf{v}_1$ $\mathbf{v}_2$ $\mathbf{v}_3$ $\mathbf{v}_4$

$\mathbf{v}_5$

Reweight / Linear Reprocess

$\mathbf{y}_1$ $\mathbf{y}_2$ $\mathbf{y}_3$ $\mathbf{y}_4$

$\mathbf{y}_5$

$$\begin{aligned} \mathbf{v}_1^T \cdot \mathbf{v}_1 &= w_{11}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_2 &= w_{12}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_3 &= w_{13}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_4 &= w_{14}^* \\ \mathbf{v}_1^T \cdot \mathbf{v}_5 &= w_{15}^* \end{aligned}$$

Normalize

$$\begin{aligned} w_{11} \\ w_{12} \\ w_{13} \\ w_{14} \\ w_{15} \end{aligned}$$

$$\text{where } w_{11} + w_{12} + w_{13} + w_{14} + w_{15} = 1$$

$$w_{11} \cdot \mathbf{v}_1 + w_{12} \cdot \mathbf{v}_2 + w_{13} \cdot \mathbf{v}_3 + w_{14} \cdot \mathbf{v}_4 + w_{15} \cdot \mathbf{v}_5 = \mathbf{y}_1$$

(Reweighs all vector towards $\mathbf{v}_1$ so it has context of other words' embedding)

Similarly:

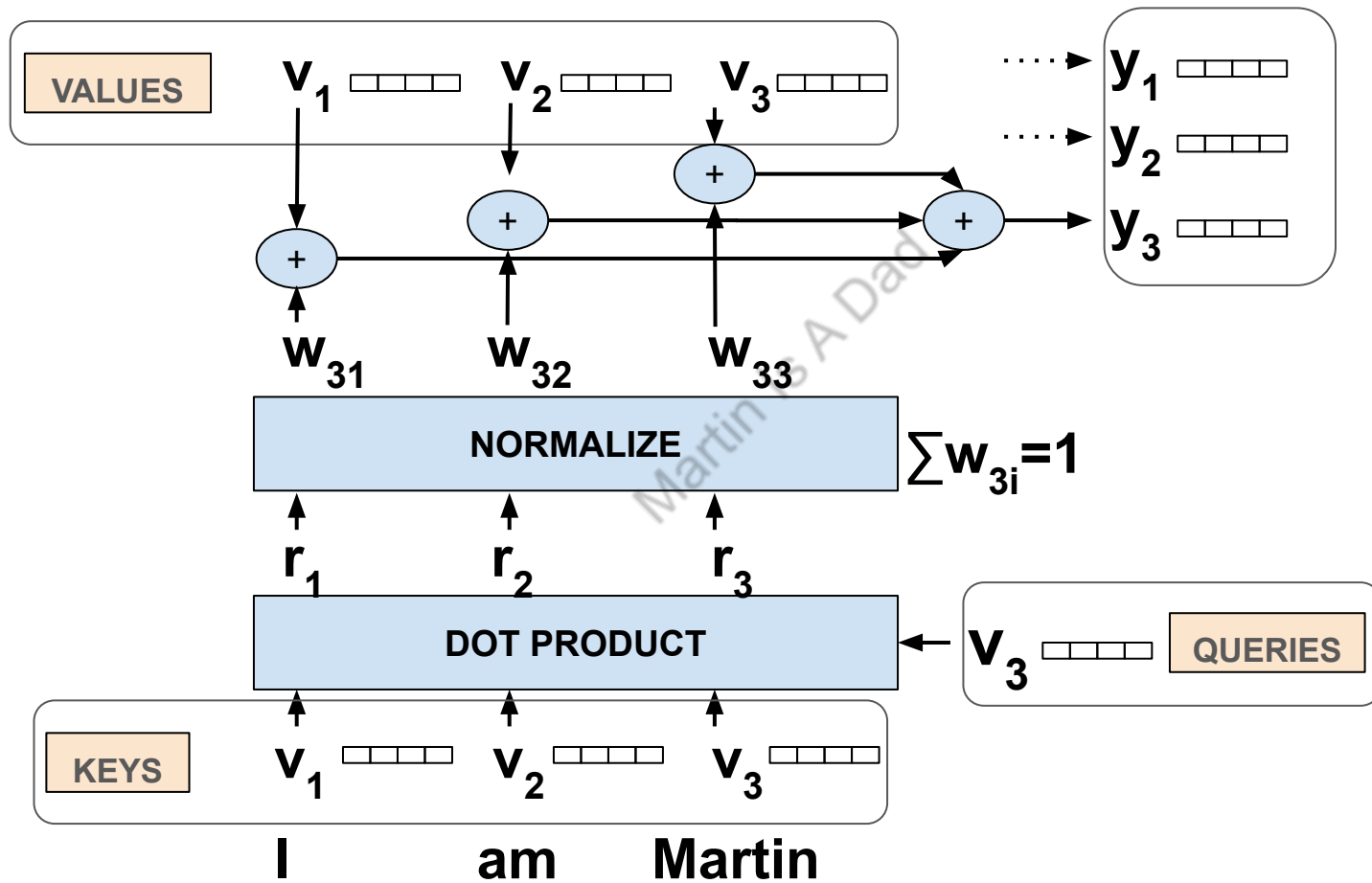
$$w_{21} \cdot \mathbf{v}_1 + w_{22} \cdot \mathbf{v}_2 + w_{23} \cdot \mathbf{v}_3 + w_{24} \cdot \mathbf{v}_4 + w_{25} \cdot \mathbf{v}_5 = \mathbf{y}_2$$

$$w_{31} \cdot \mathbf{v}_1 + w_{32} \cdot \mathbf{v}_2 + w_{33} \cdot \mathbf{v}_3 + w_{34} \cdot \mathbf{v}_4 + w_{35} \cdot \mathbf{v}_5 = \mathbf{y}_3$$

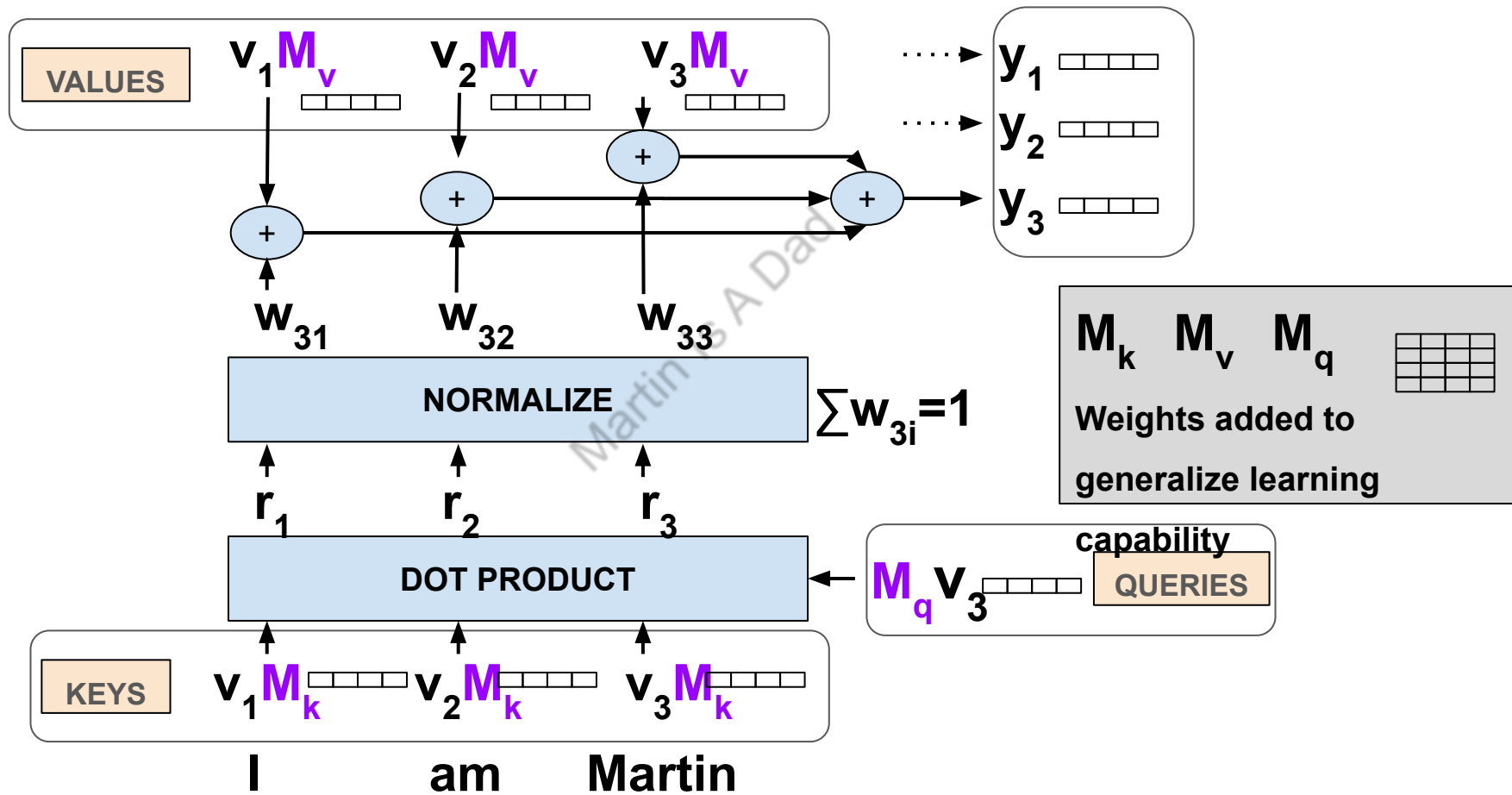
$$w_{41} \cdot \mathbf{v}_1 + w_{42} \cdot \mathbf{v}_2 + w_{43} \cdot \mathbf{v}_3 + w_{44} \cdot \mathbf{v}_4 + w_{45} \cdot \mathbf{v}_5 = \mathbf{y}_4$$

$$w_{51} \cdot \mathbf{v}_1 + w_{52} \cdot \mathbf{v}_2 + w_{53} \cdot \mathbf{v}_3 + w_{54} \cdot \mathbf{v}_4 + w_{55} \cdot \mathbf{v}_5 = \mathbf{y}_5$$

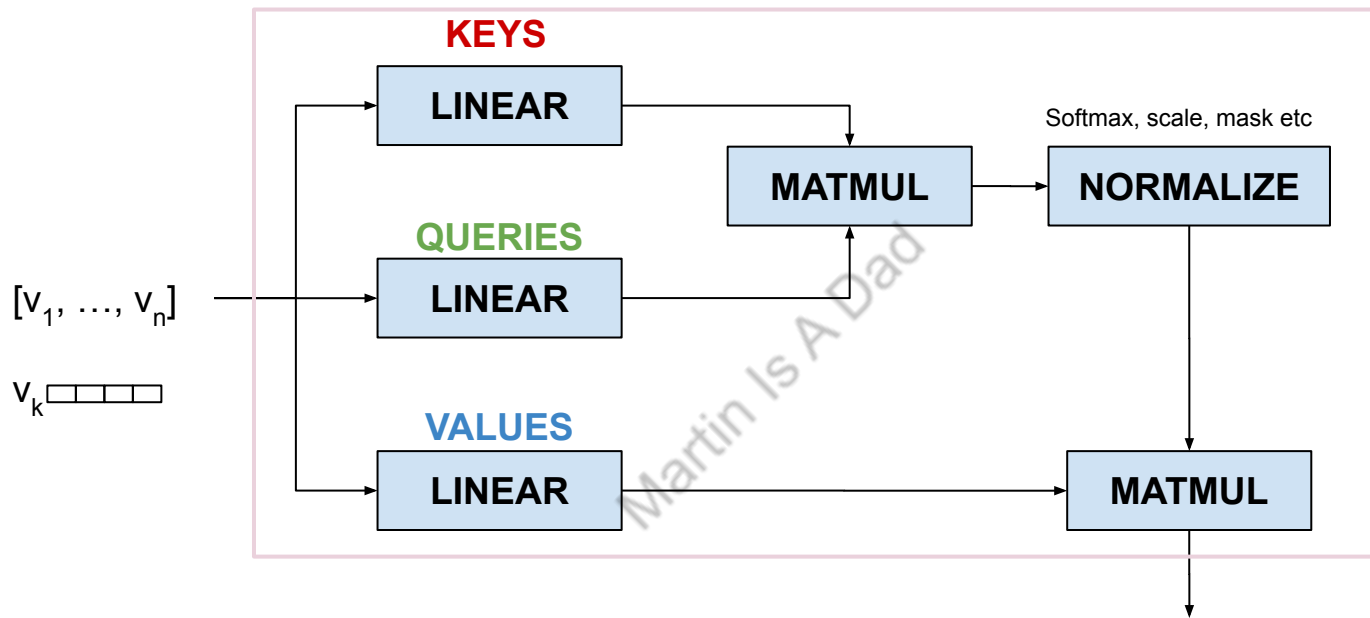
Attention: Keys, Values, Queries



Attention: Keys, Values, Queries



Attention: Keys, Values, Queries



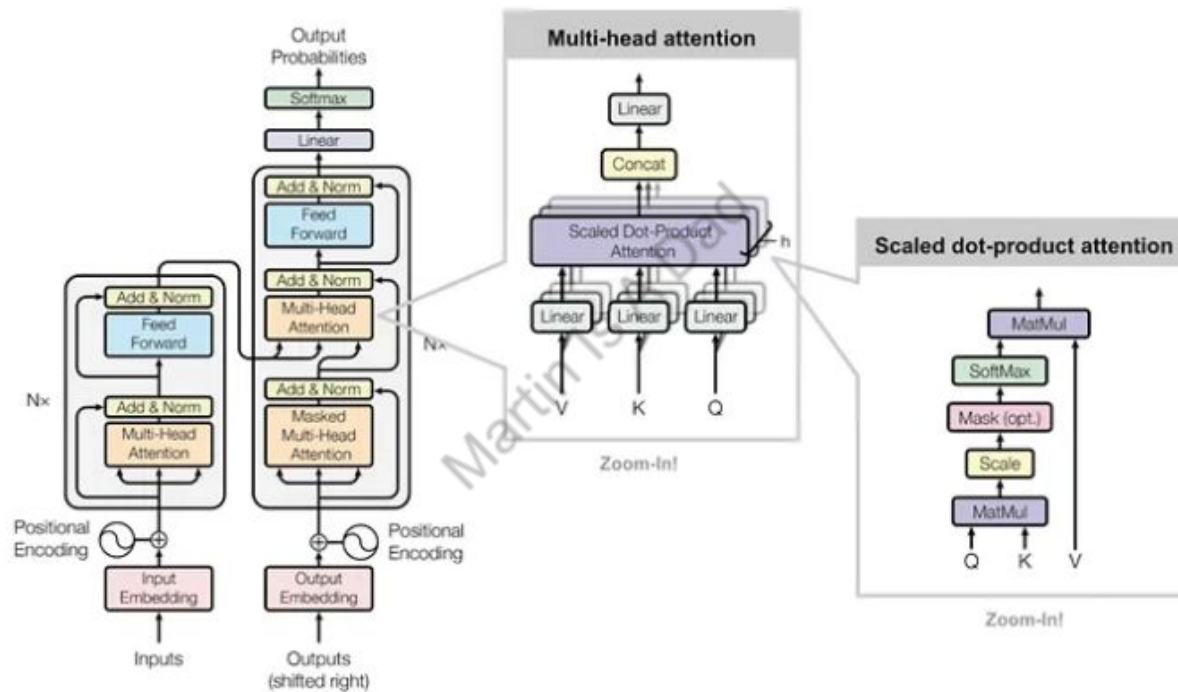
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \mathbf{V} \in \mathbb{R}^{m \times d_v}$$

$[y_1, \dots, y_n]$

y_k

--	--	--	--

Attention: Keys, Values, Queries



Attention: Keys, Values, Queries

Q: Why scale?

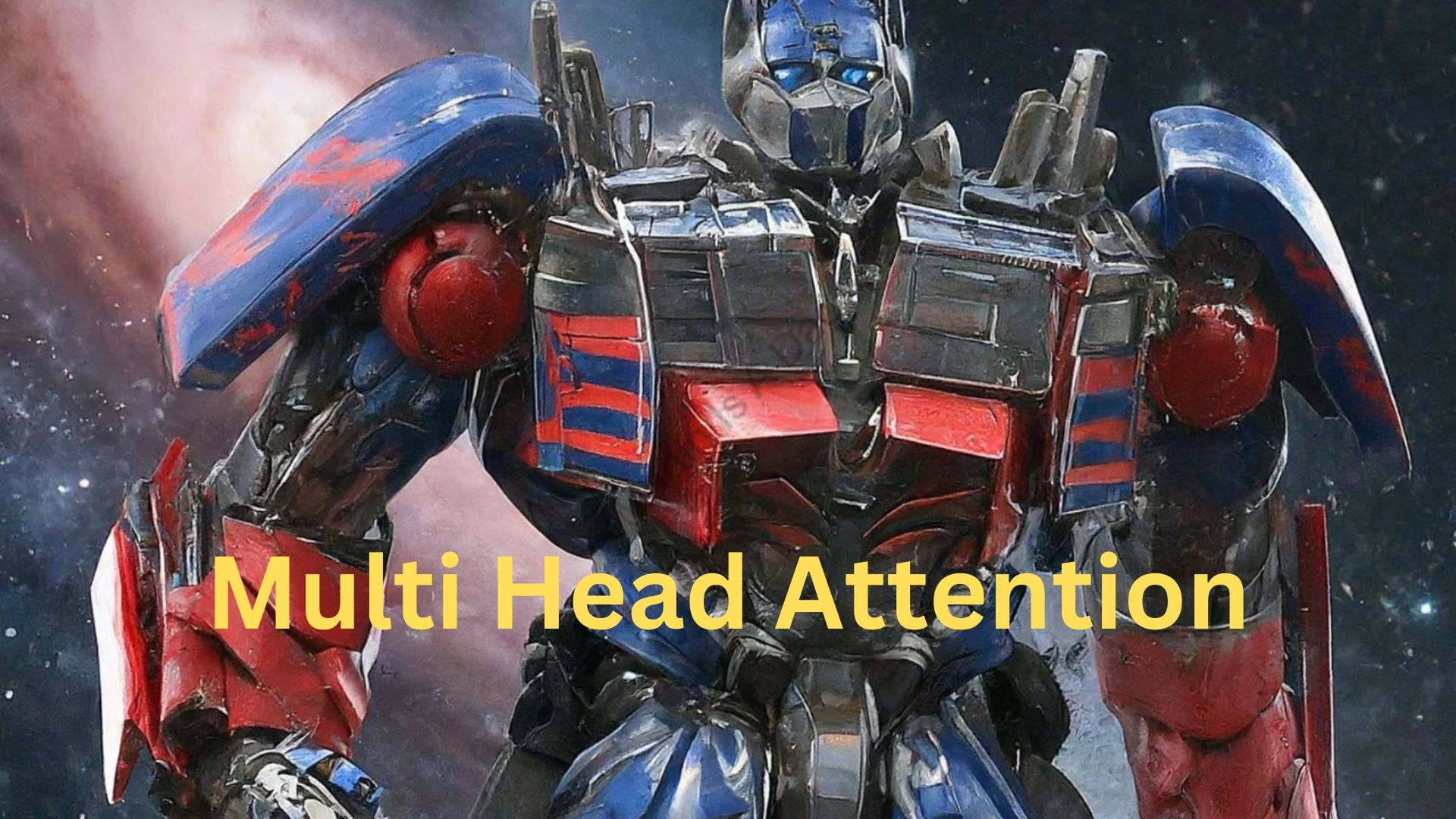
A: Key, Query vector could be large, dot product could be disproportionately large, this can lead to vanishing gradients or exploding gradients that hurt training.

Q: Why $\sqrt{d_k}$ to be specific?

A: Mean 0, variance is standard normal distribution. If we assume Q and K are d_k dimensional vectors whose components are independent random variables that follow standard normal distribution, $q \cdot k = \sum_{i=1}^{d_k} u_i v_i$ their dot product will have mean 0 and variance d_k . Since we prefer it to be standard normal distribution (variance 1) so the values are not too scattered, we divide by $\sqrt{d_k}$.

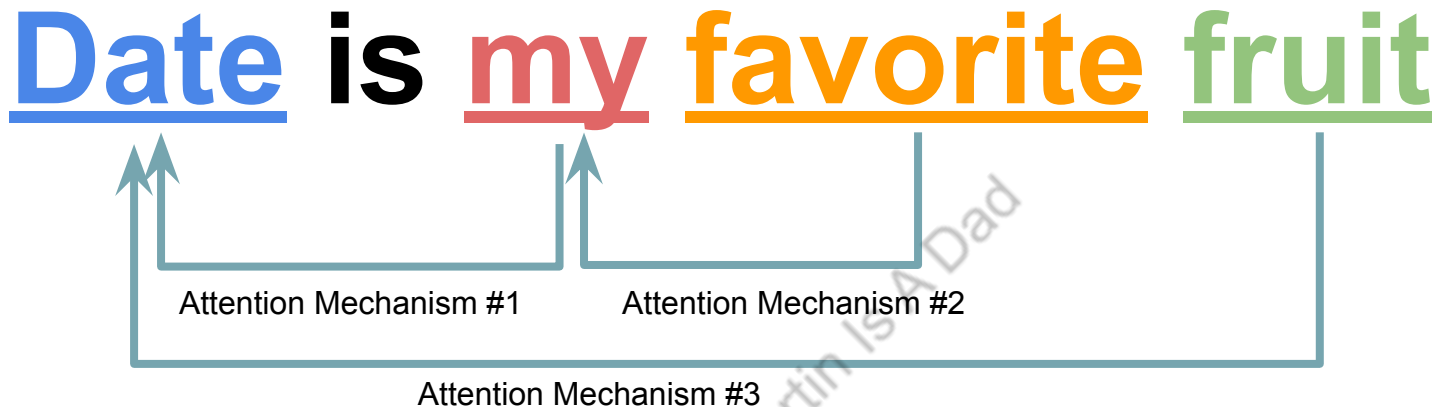
Q: Why softmax?

So we have the attention scores transformed accordingly to probability distribution that sums to 1.

A detailed image of the Optimus Prime robot from the Transformers franchise. He is shown from the waist up, facing forward. His body is primarily blue and red, with a silver face and chest. He has a complex, mechanical design with many joints and panels. The background is a dark, starry space with a bright, hazy light source on the left. The text "Multi Head Attention" is overlaid in the center in a bold, yellow font.

Multi Head Attention

Attention for natural language



- Attention mechanism can capture context and relationship between tokens. Natural language is complex and usually will have multiple aspects of context / relationship
- For example, we might need to capture {date $\leftrightarrow$ fruit}, {date $\leftrightarrow$ my}, {favorite $\leftrightarrow$ my} context
- We can use different set of parameters to capture different types of context / relationships

Attention for Computer Vision

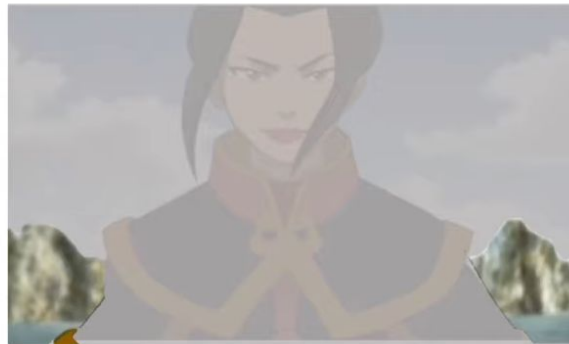
Attention Filter 1



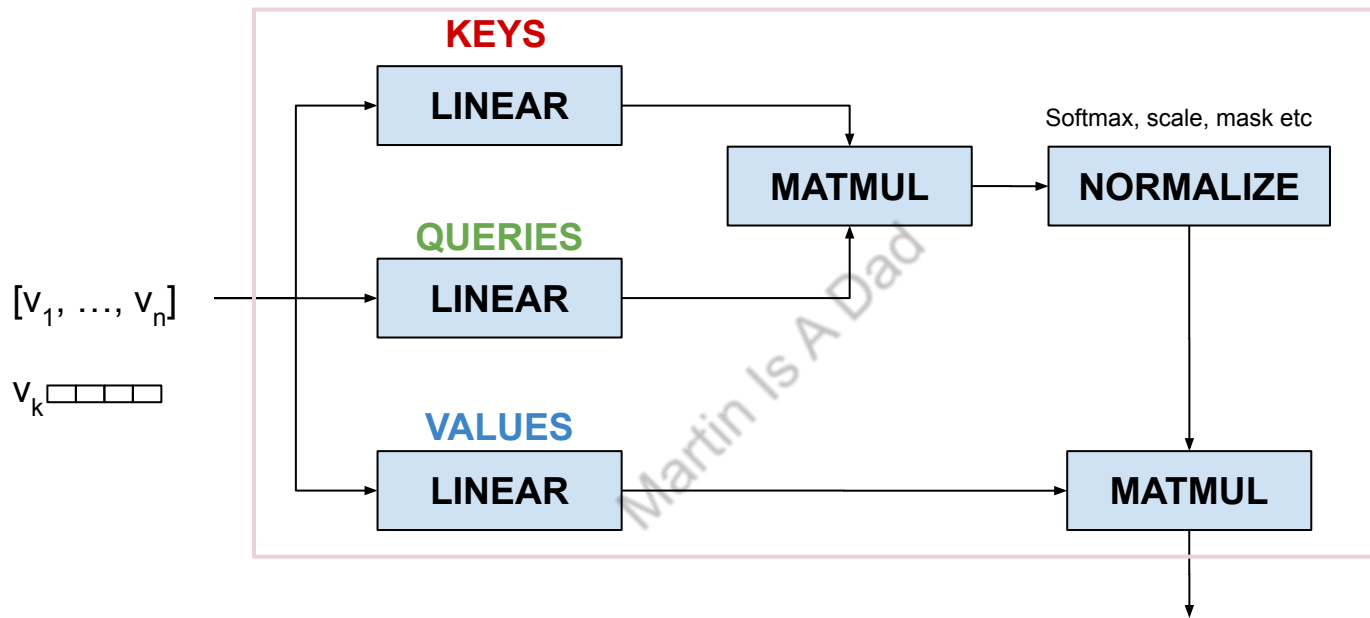
Attention Filter 2



Attention Filter 3



Attention Block



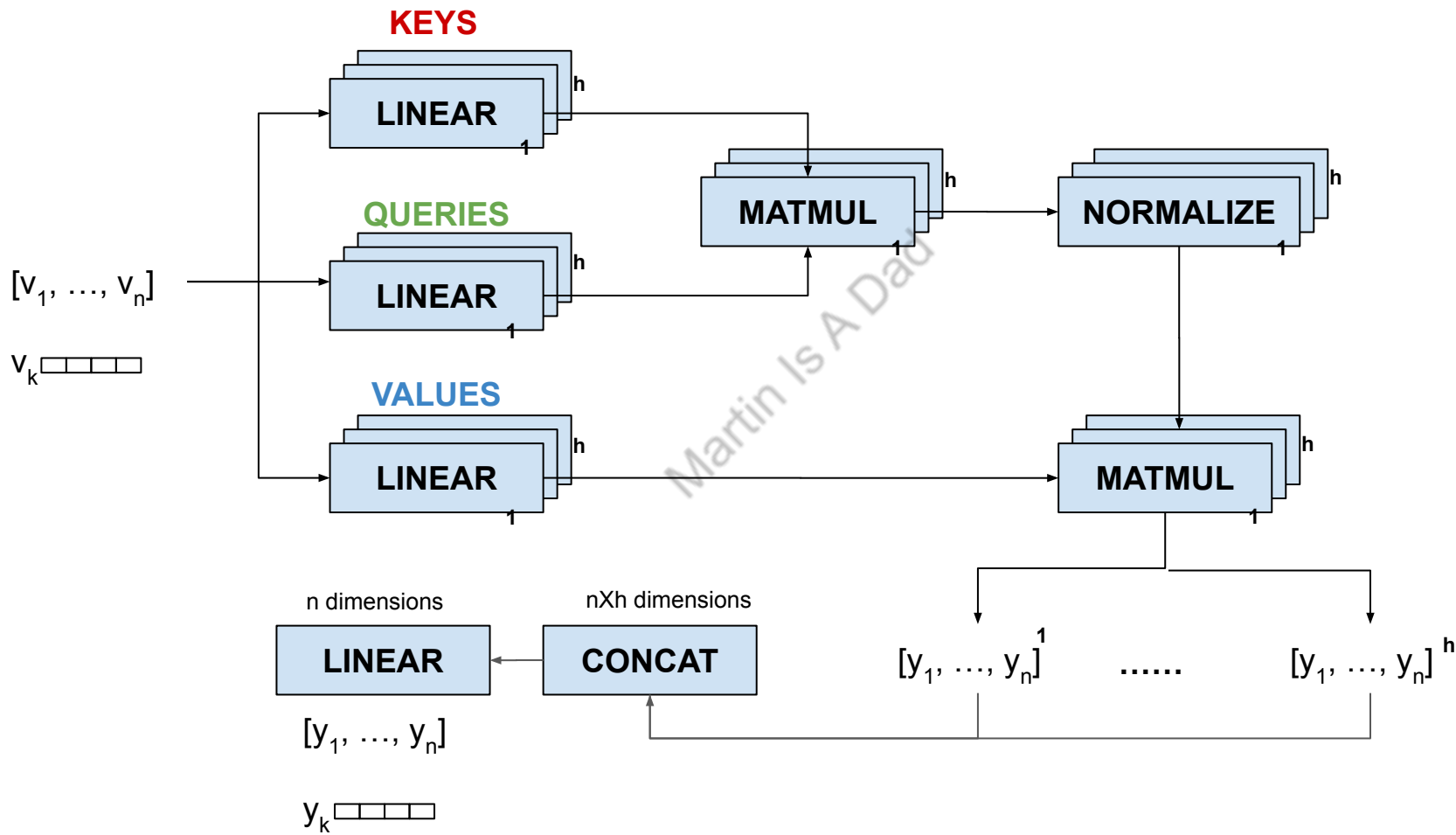
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \mathbf{V} \in \mathbb{R}^{m \times d_v}$$

$[y_1, \dots, y_n]$

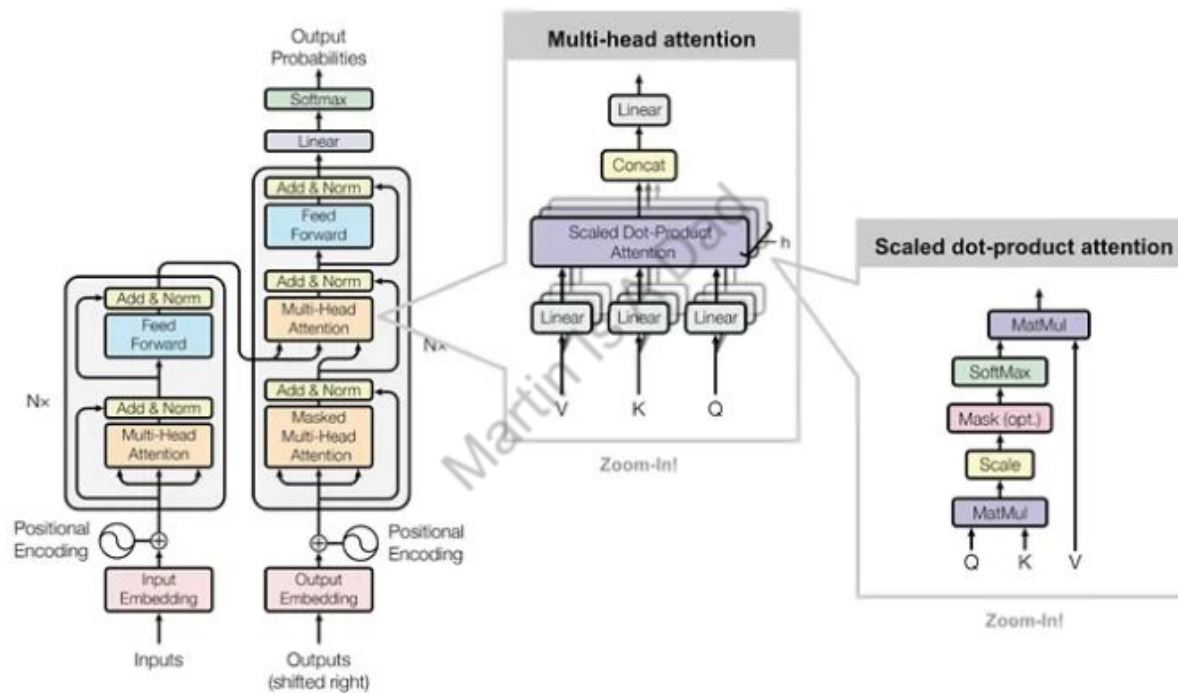
y_k

--	--	--	--

Multi Head Attention Block

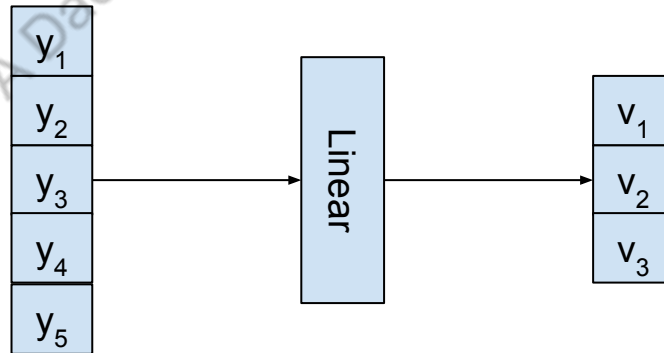
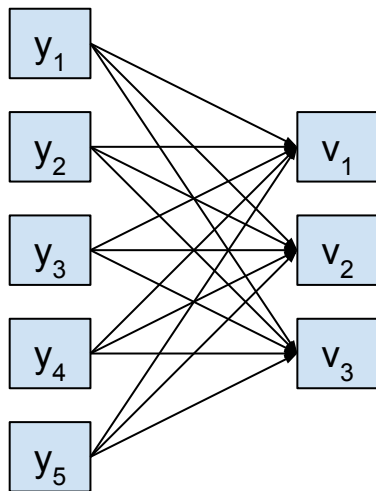


Multi Head Attention Block



Linear / Dense / Fully Connected Layer

1. Maps input to output
2. Change vector's dimension





KV Cache

Transformer: Training vs Inference

- Transformer behaves **differently** during training and inference phase. The main difference occurs in the decoder (grey box on the right half).
- During training we know the entire output sentence and can pass it to the decoder all at once. During inference (say translation of `我是马丁` to `I am Martin`) the decoder must generate one word at a time in an autoregressive manner. The decoder uses previously predicted word to help predict next word.
- Masked attention module will make sure each token in a sequence only attends to previous tokens and itself, not future tokens.

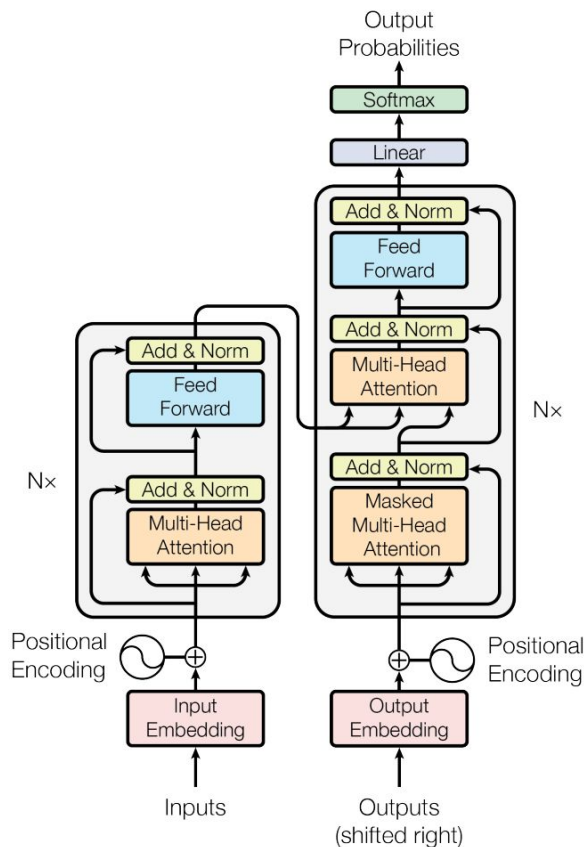


Figure 1: The Transformer - model architecture.

Transformer: Inference

	Encoder Input	Decoder Input	Output
Step 1	I am Martin	Start	我
Step 2		我	是
Step 3		我是	马
Step 4		我是马	了

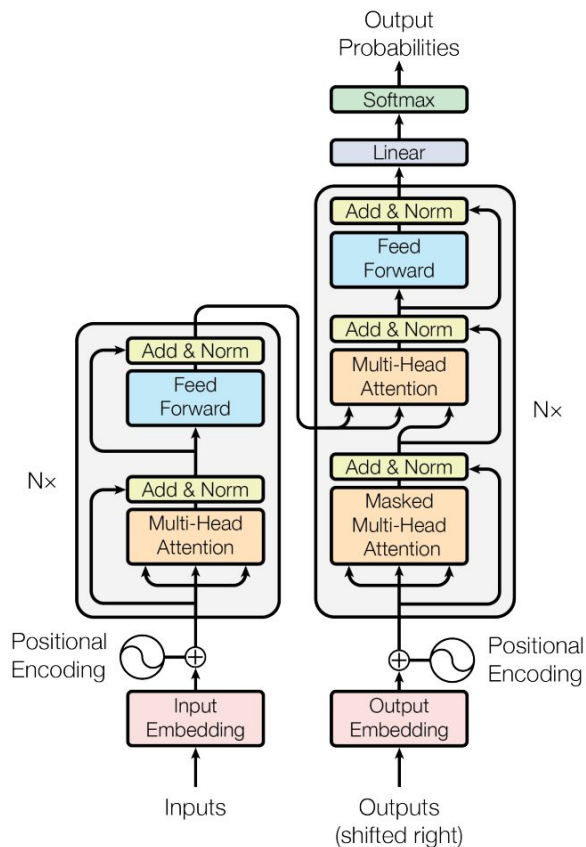
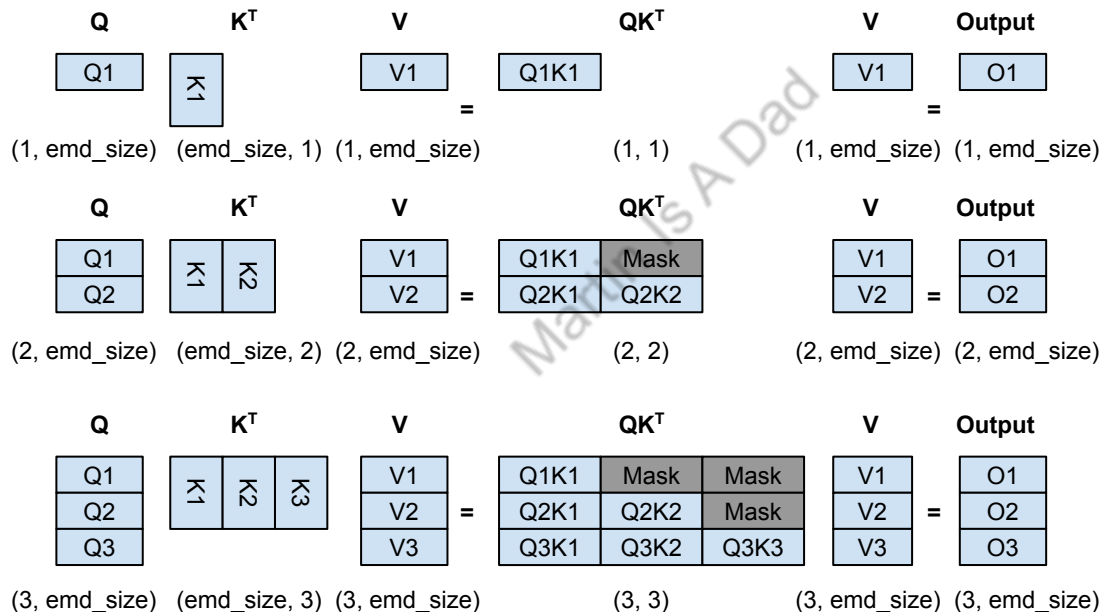


Figure 1: The Transformer - model architecture.

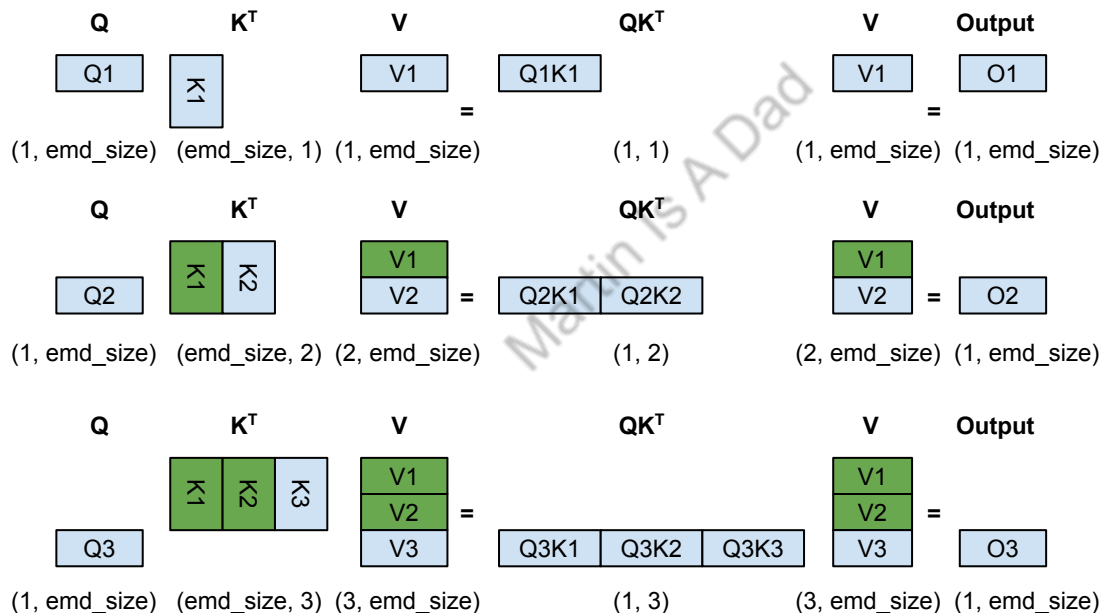
Transformer: Inference without Cache

During inference's autoregression, at each generation step we are re-calculating the same previous token attention.



Transformer: Inference with Cache

During inference's autoregression, at each generation step we are re-calculating the same previous token attention.



Transformer: KV Cache Tradeoff

(+): Number of FLOPs (floating point operation per second) drop

- Drops from $O(t^2)$ to $O(t)$, t is number of sequence

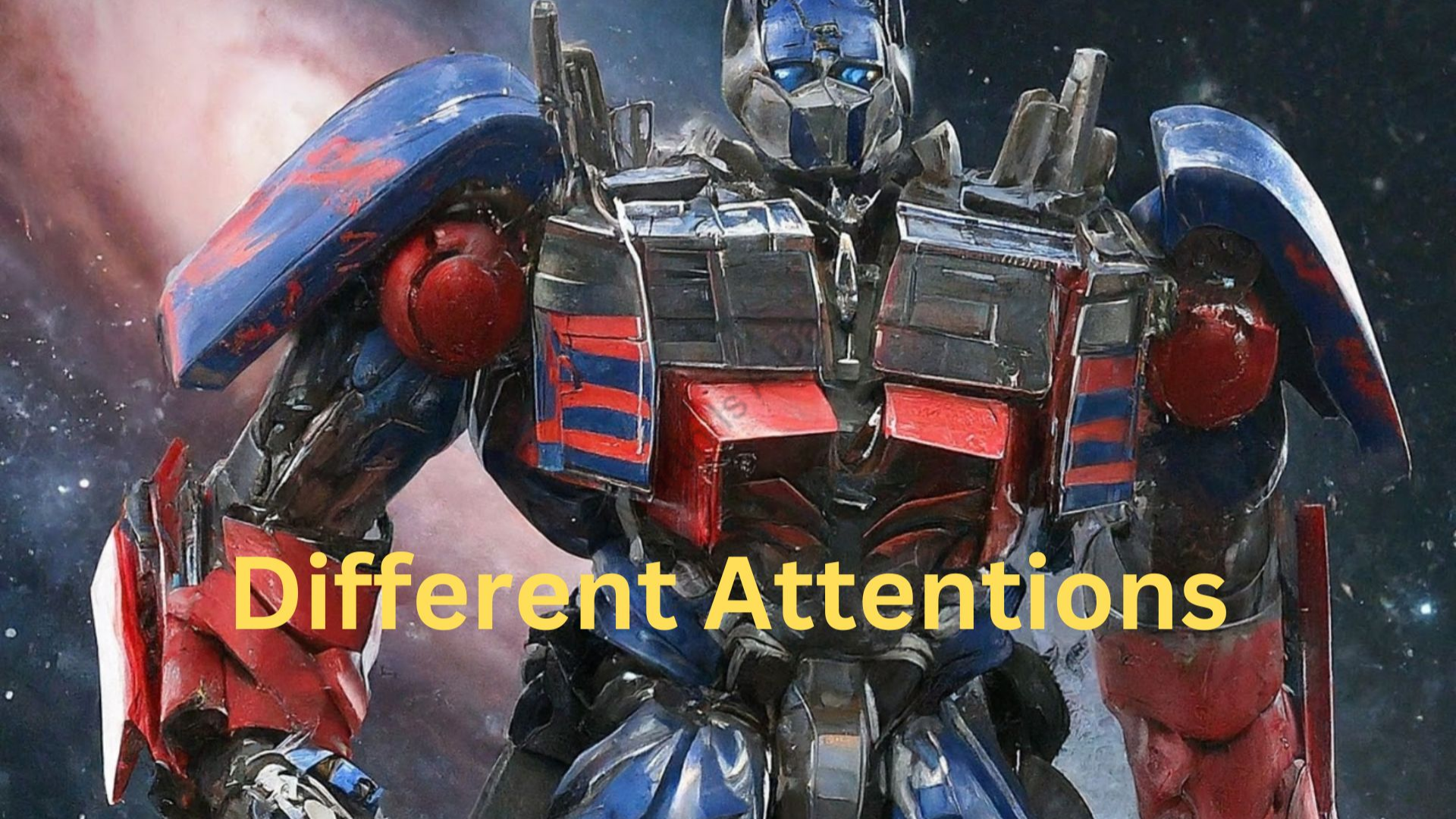
(-): Extra memory needed.

- The memory amount is of size $[2, \text{num_of_tokens}, \text{num_layers}, \text{num_kv_heads}, \text{kv_dim}]$, 2 as in key and value

Transformer: KV Cache Eligibility

New token's attention calculation only depend on itself and previous tokens

Martin Is A Dad



Different Attentions

Different Attentions

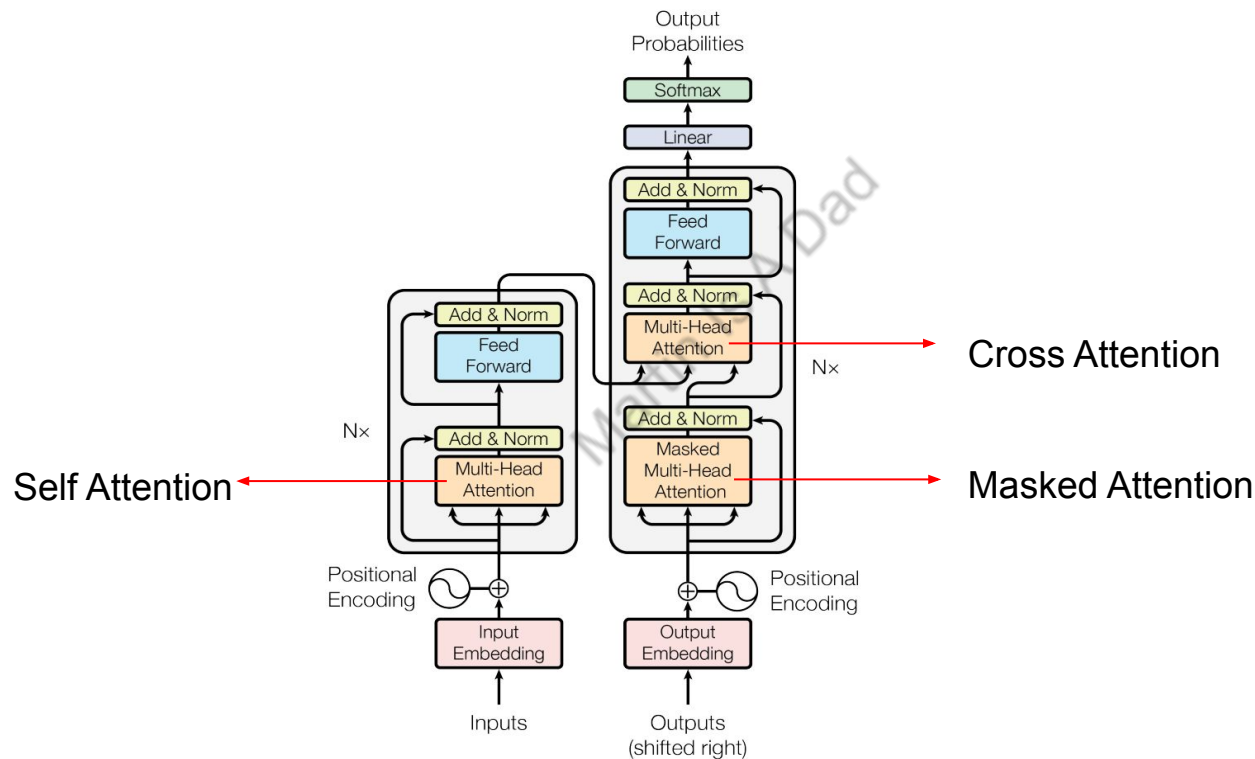
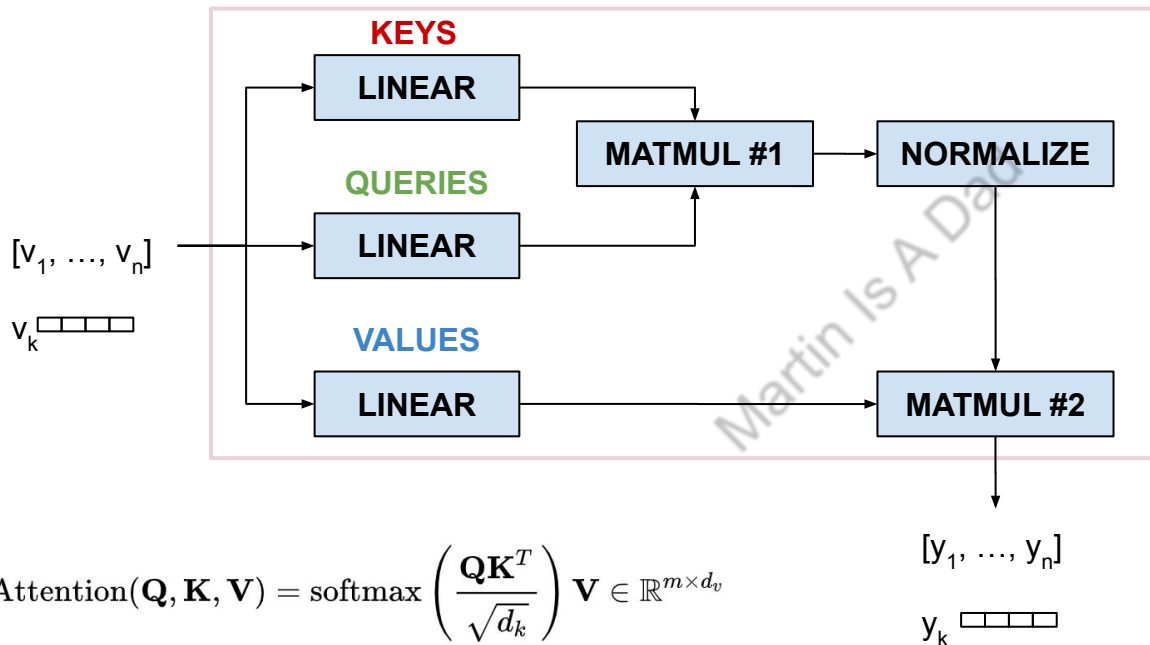


Figure 1: The Transformer - model architecture.

Attention Mechanism Intuition



$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \in \mathbb{R}^{m \times d_v}$$

- **MATMUL #1:** Dot product cosine similarity between **KEY** and **QUERY**
- **NORMALIZE:** softmax to map similarities to probability, scale to prevent gradient diminishing/explosion
- **MATMUL#2:** Apply the similarity distribution (post softmax) to weight **VALUE**

Basic Attention

External Query

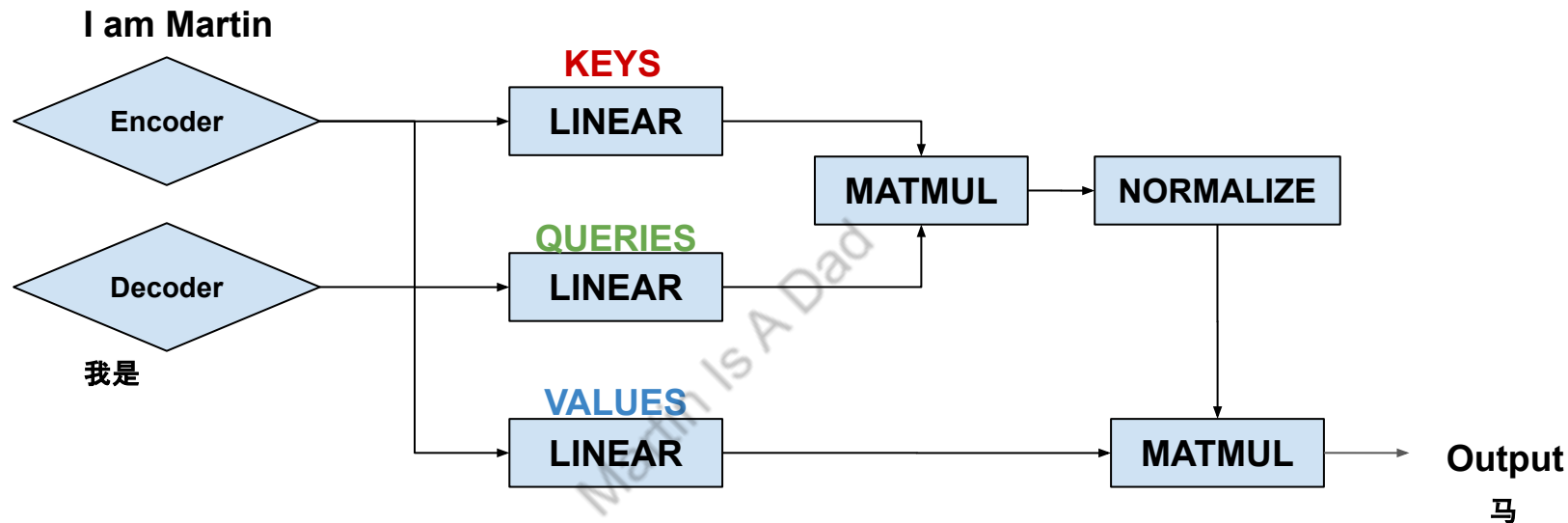
		I	have	a	white	cat
Key	I	1	0.5	0.3	0.2	0.9
	love	0.2	0.6	0.2	0.1	0.8
	little	0.4	0.9	0.1	0.3	0
	animals	0.7	1	0	0.8	0.1

Self Attention

Query = Key

	I	love	little	animals
I	1	0.5	0.3	0.2
love	0.2	0.6	0.2	0.1
little	0.4	0.9	0.1	0.3
animals	0.7	1	0	0.8

Cross Attention



- Say we are translating [I am martin] to Chinese, and we already generated the sequence [我是]
- Cross attention is to establish relationships between input sequence and generated output sequence. So the attention cosine similarity happens between Query (decoder) and Key (encoder), and the similarity will apply weight to Value (encoder)

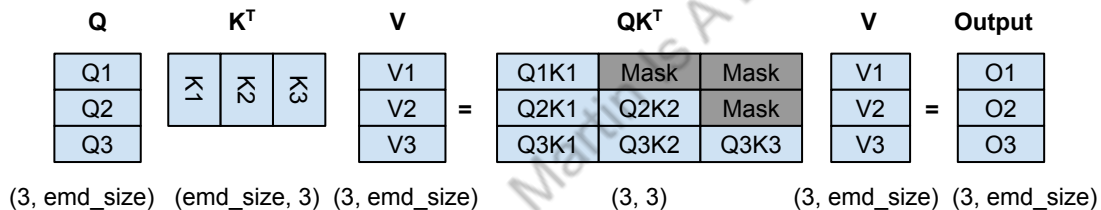
Cross Attention

		Encoder		
		I	am	Martin
Decoder	我	1	0.5	0.3
	是	0.2	0.6	0.2

Masked Attention

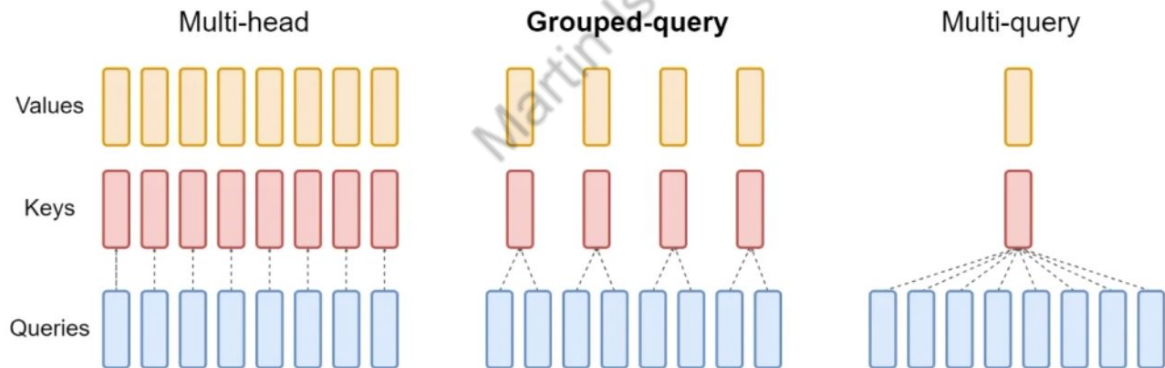
Causal mask is applied to Attention mechanism to ensure a model predicts each token in a sequence solely based on the previous tokens, not on any future tokens.

This enables KV Cache to speed up Transformer.



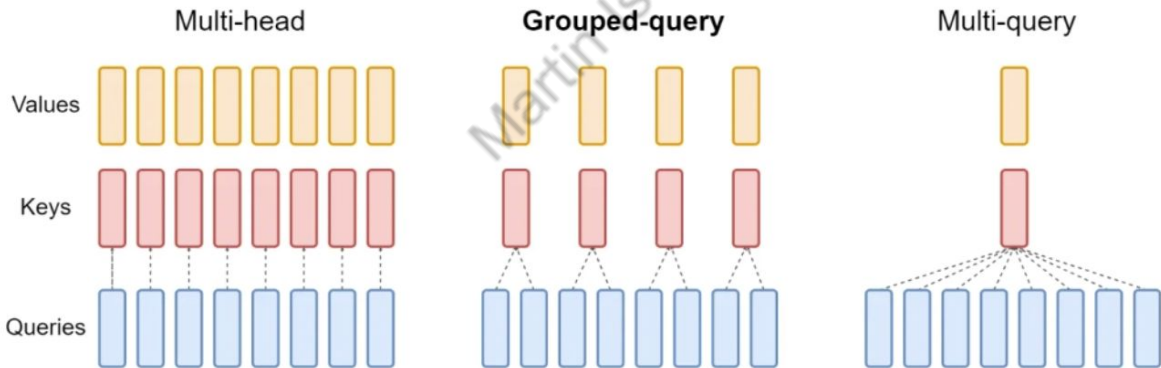
Multi Query Attention

- KV Cache memory size becomes bottleneck, especially when the sequence is longer and longer. MQA & GQA is to address the concern.
- MQA is to share a single key and a single value head across all query heads, which can significantly reduce memory usage but will also impact the accuracy of attention.



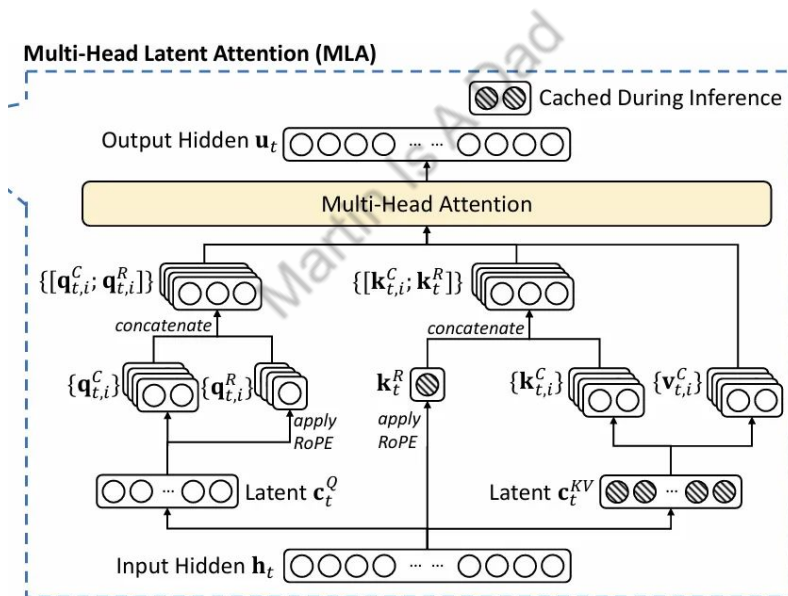
Grouped Query Attention

- KV Cache memory size becomes bottleneck, especially when the sequence is longer and longer. MQA & GQA is to address the concern.
- GQA can be seen as an interpolating method between MHA and MQA, where a single pair of key and value heads will be shared only by a group of query heads, not all queries. The quality impact is less.



Multihead Latent Attention

DeepSeek uses a low rank attention mechanism they call “Multi-Head Latent Attention” which projects the keys and values into a shared low-rank space. This equates to reducing the sum of the dimensions of the query, key, and value vectors to below the hidden dimension.





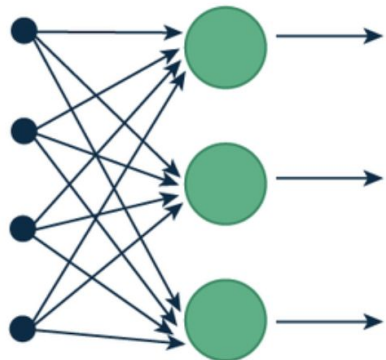
Many Many Variances

A detailed image of Optimus Prime, the leader of the Autobots, from the Transformers franchise. He is shown from the waist up, wearing his iconic blue and red armor. His chest features a red and blue striped pattern. He has a determined expression on his face. The background is a dark, starry space with a bright, hazy light source on the left.

Feed Forward Network

Why Named Feed Forward?

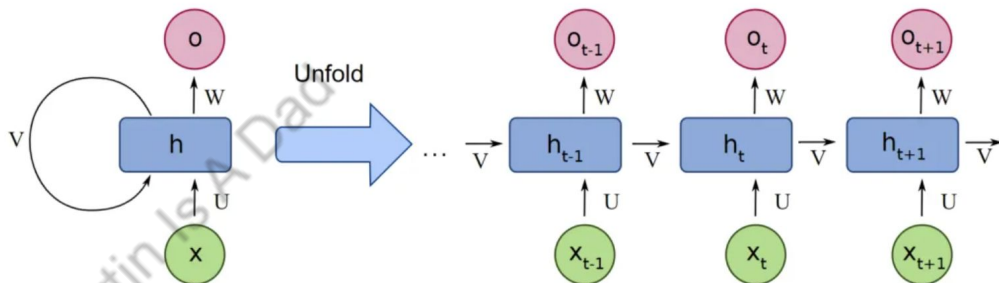
Feed Forward Neural Network



Information flows in 1 direction with no circle

Good for tasks that don't require memory of past data, say image recognition

Recurrent Neural Network

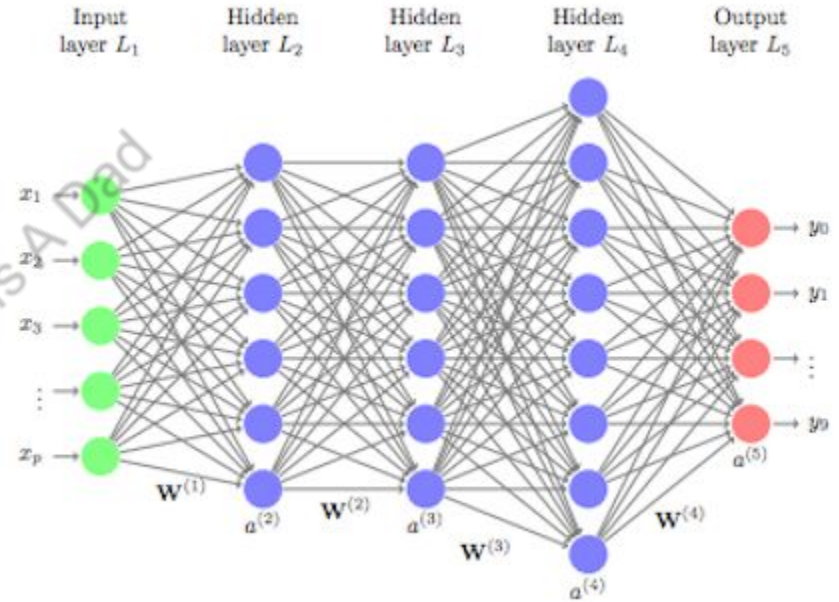


Processes information in loops (allow information from previous steps to be fed back into the network)

Good for sequential data, say text recognition, translation

Feed Forward Architecture

- Input Layer: first layer of the neural network
- Output Layer: last layer of the neural network
- Hidden Layer: every layer between input and output layers



FFN Inside Transformer

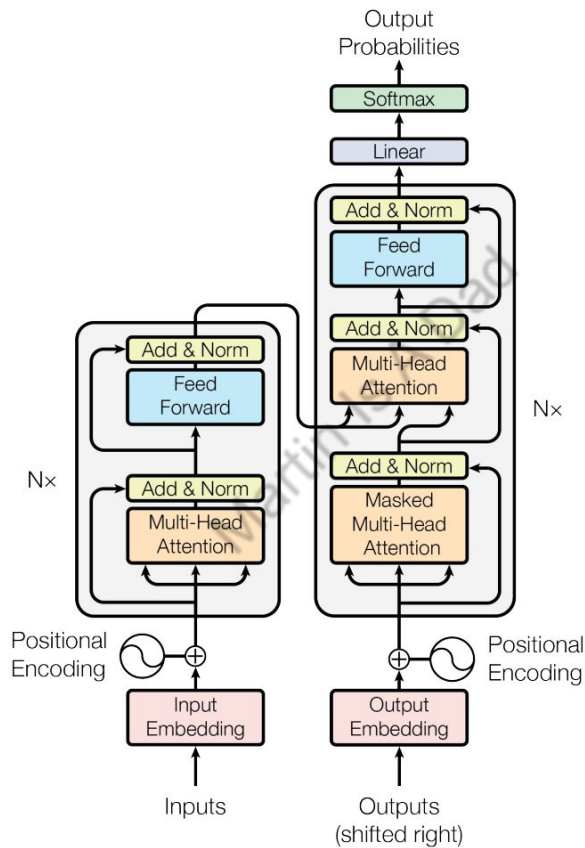


Figure 1: The Transformer - model architecture.

FFN Inside Transformer: Basics

- Formula: $\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$
- TLDR: Two linear transformations with a ReLU activation function in between
 - ReLU: A non-linear activation function used in neural networks. $\text{ReLU}(x) = \max(0, x)$
- Intuition (thanks Gemini):
 - **Non-linearity**: The first fully connected layer projects the input to a higher dimensional space (allowing the network to learn more complex relationships), and the go through the activation function applied between the layers (allowing the network to learn non-linear complex relationships).
 - **Dimensionality reduction**: The second fully connected layer then projects the data back to its original dimensionality, ensuring compatibility with the next layer.
 - **Parallel processing**: Since each position in the sequence is processed separately through the FFN, the computation can be parallelized efficiently.

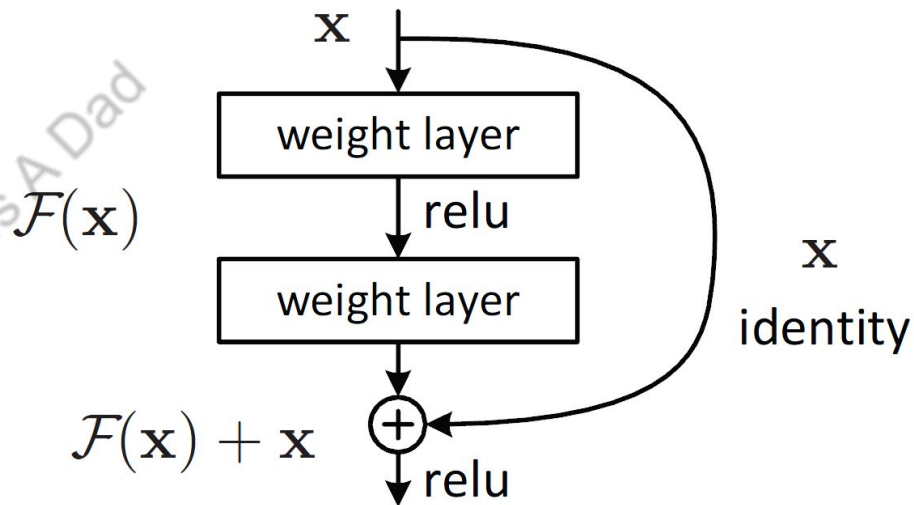
FFN Inside Transformer: More

Dropout

- A regularization technique used to prevent overfitting by randomly "dropping out" (deactivating or set to 0s) a certain percentage of neurons during training, forcing the network to learn more robust features and not rely too heavily on any single neuron, thus improving its ability to generalize to new data.
- Advantages
 - **Reduces overfitting:** By preventing neurons from co-adapting too much, dropout helps the network learn more generalized features.
 - **Improved generalization:** By forcing the network to learn features that are not dependent on specific neurons, dropout improves the model's performance on unseen data.

Add & Norm: Residual Connection

- **Goal:** mitigate the vanishing gradient problem for deep neural networks. Originally from ResNet, now used more in deep neural networks, say Transformer. This enable deeper architectures, by resolving the vanishing gradient problem, which is more serious as layers increase.
- **How:** allowing gradients to bypass intermediate layers. Adding identity function with the reweighted values.



Add & Norm: Residual Connection

- **Goal:** mitigate the vanishing gradient problem for deep neural networks. Originally from ResNet, now used more in deep neural networks, say Transformer. This enable deeper architectures, by resolving the vanishing gradient problem, which is more serious as layers increase.
- **How:** allowing gradients to bypass intermediate layers. Adding identity function with the reweighted values.

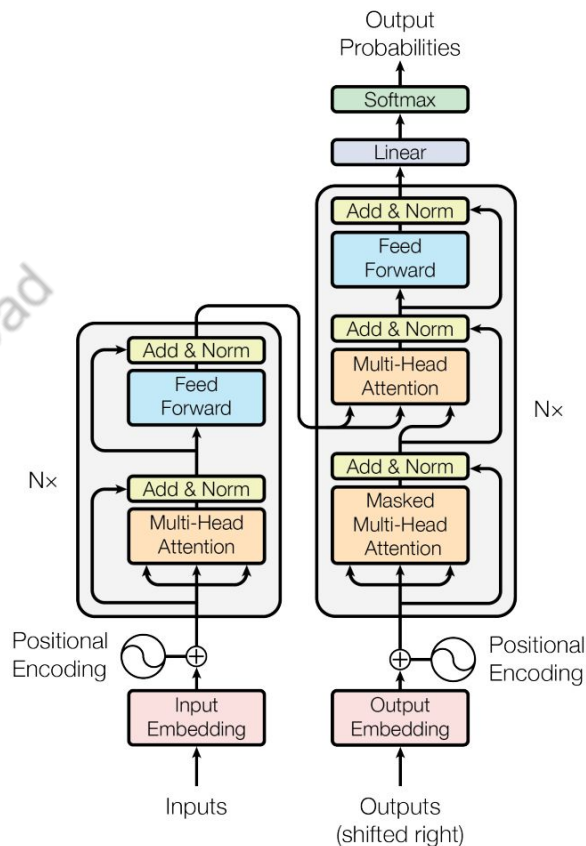


Figure 1: The Transformer - model architecture.

Add & Norm: Layer Normalization

Layer normalization is a crucial technique in transformer models that helps stabilize and accelerate training by normalizing the inputs to each layer. It ensures that the model processes information consistently, regardless of the input's scale or distribution.

More in next episode!

Martin Is A Dad



Normalizations

What is Normalization

Wiki: Normalization means adjusting values measured on different scales to a notionally common scale

Min Max Normalization	Z Score Normalization
$z = \frac{x - \min(x)}{\max(x) - \min(x)}$	$z = \frac{x - \mu}{\sigma}$ μ = Mean σ = Standard Deviation
Only re-scale	• Re-scale • Re-center

Why Normalization In Deep Learning?

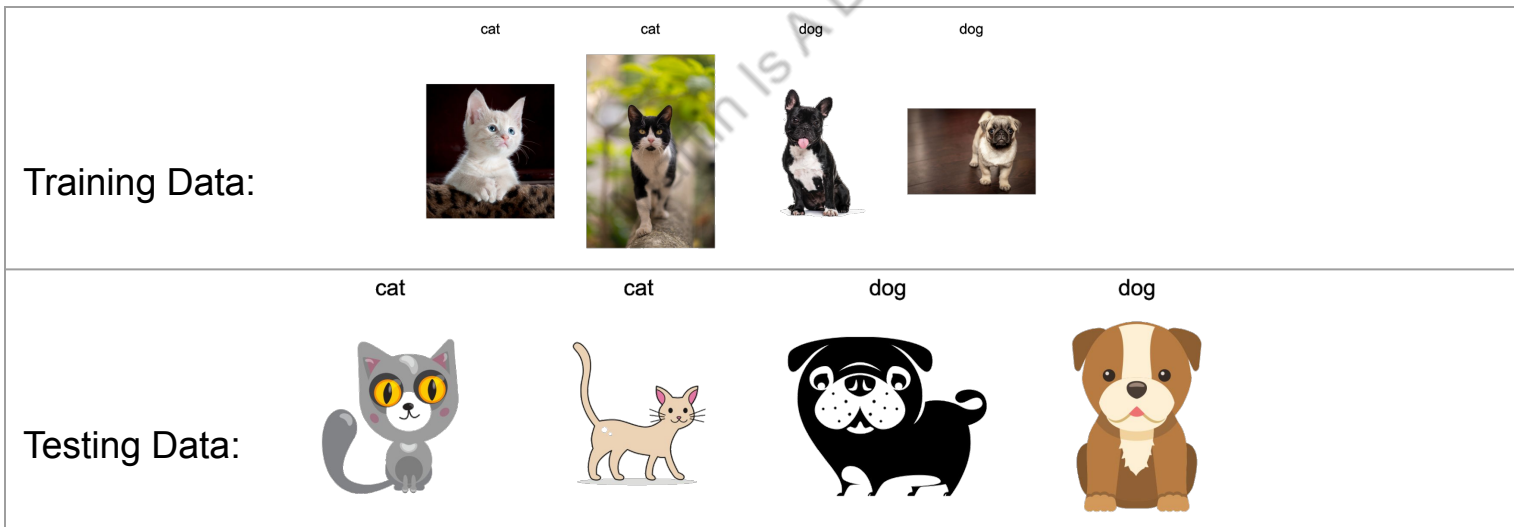
It helps stabilize the training process by ensuring features are on a similar scale, preventing features with larger values from dominating the learning process, which leads to faster convergence and improved model performance

- **Faster convergence:** By scaling features to a similar range, the gradient descent optimization algorithm can update weights more effectively, leading to quicker training times.
- **Improved generalization:** Normalization can help the model generalize better to unseen data by reducing the sensitivity to feature scales.
- **Stability in gradient calculations:** When features have vastly different scales, it can lead to unstable gradients during backpropagation; normalization helps mitigate this issue.
- **Reduced internal covariate shift:** Especially with Batch Normalization, normalizing activations within each layer helps maintain a stable distribution throughout the network, which is crucial for deep architectures.

Why Normalization In Deep Learning?

Reduced internal covariate shift: Especially with Batch Normalization, normalizing activations within each layer helps maintain a stable distribution throughout the network, which is crucial for deep architectures.

When the input data's distribution differs from training dataset, or as it moves through the layers of your network, this can confuse the model. For example (taken from d2l.ai)



Where to Apply Normalization in Deep Learning

- **Input Data:** The data that you feed into the neural network (e.g., features like f_1 , f_2 , f_3) can be normalized before entering the network. This step is similar to traditional machine learning's pre-processing
- **Activations:** You can also normalize the activations (outputs) from hidden layers in the neural network. This is often done to stabilize and accelerate training, especially in deep networks.

Martin Is A Fool

Batch Normalization

x_1

x_2

x_3

x_4

Input
Layer

Features	Sample 1 Output	Sample 2 Output	Sample 3 Output	Mean	Standard Deviation
x_1	3	2	11	5.3	4.9
x_2	5	1	8	4.7	3.5
x_3	7	10	1	6	4.6
x_4	6	3	2	3.7	2.1

Normalization happens across mini batch, independent for each feature

Batch Normalization

- For all features, apply the followings to activations, and get normalize value:

$$\text{Normalized Value} = \frac{\text{Original Value} - \mu}{\sigma}$$

- Apply scaling and shifting parameter. Each node / neuron have learning parameter gamma (γ) and beta (β)

$$\text{Final Value} = (\text{Normalized Value} \times \gamma) + \beta$$

Why is Batch Normalization Bad For Transformer?

Batch Normalization Limitations:

- In batch normalization, the mean and standard deviation are calculated within the current mini-batch. However, when the batch size is small, the sample mean and sample standard deviation are not representative enough of the actual distribution.
- For sequence models, we see more small batch size when sequence is long, to allow the model to update its parameters more frequently with smaller subsets of data. Transformer is the most popular model for sequential model.
- To be more specific, we usually do padding (adding 0s) to make the input size equal during self attention, which is not part of the original data and can mislead the data a lot.

Why is Batch Normalization Bad For Transformer?

Batch Normalization Limitations:

- To be more specific, we usually do padding (adding 0s) to make the input size equal during self attention, which is not part of the original data and can mislead the data a lot.

Hello	3	2	11
!	5	1	8
Padding	0	0	0
Padding	0	0	0
My	3	2	11
Name	3	2	11
Is	3	2	11
Martin	3	2	11

Layer Normalization

x_1

x_2

x_3

x_4

Input
Layer

Features	Sample 1 Output	Sample 2 Output	Sample 3 Output
x_1	3	2	11
x_2	5	1	8
x_3	7	10	1
x_4	6	3	2
Mean	5.25	4	5.5
Standard Deviation	1.7	4	4.8

Normalization happens across features, independent for each sample

Layer Normalization

- For all features, apply the followings to activations, and get normalize value:

$$\text{Normalized Value} = \frac{\text{Original Value} - \mu}{\sigma}$$

- Apply scaling and shifting parameter. Each node / neuron have learning parameter gamma (γ) and beta (β)

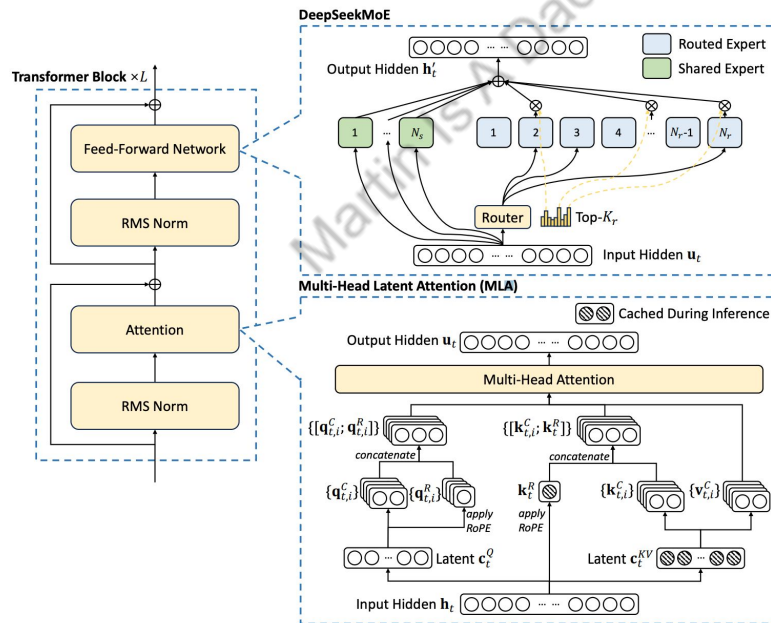
$$\text{Final Value} = (\text{Normalized Value} \times \gamma) + \beta$$

RMS (Root Mean Square) Normalization

- RMSNorm, normalizes the activations by dividing them by the root mean square (RMS) of the activations for each layer. Unlike LayerNorm, RMSNorm typically does not center the activations (subtract the mean) before normalization. It does include scaling by a learnable parameter (similar to γ in LayerNorm), but there's usually no shifting parameter (β). The RMS is calculated as the square root of the mean of the squares of the activations.
- Compared with BatchNorm and LayerNorm, RMSNorm's normalization process is simple and without overhead of mean and variance calculation. Additionally, RMSNorm claimed the scaling factor is more important for stabilizing the norm of the gradients than the shifting factor.

RMS (Root Mean Square) Normalization

- RMSNorm is preferred in scenarios where computational efficiency matters, to avoid the overhead of computing the exact mean and variance. Many LLMs are using this, for example LLaMa, DeepSeek etc due to its efficiency.



RMS (Root Mean Square) Normalization

$$\bar{a}_i = \frac{a_i}{\text{RMS}(\mathbf{a})} g_i, \quad \text{where } \text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}$$

- $\mathbf{a}_i$: activation of i-th neuron
- $\mathbf{g}_i$: the scaling parameter re-scale the standardized summed inputs, learned parameter for each neuron
- n : the total number of features in the input vector of this specific layer.